School of Electrical Engineering
and Telecommunications
The University of New South Wales

Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

by

Jordan Mohindra

Student ID: z5418455

Thesis submitted as a requirement for the degree
Bachelor of Engineering (Electrical Engineering)

Submitted: November 2026

Supervisor: A/Prof. Beena Ahmed

Abstract

High-resolution cascaded MIMO radar produces more raw data than a vehicle's network and central processor can comfortably absorb. Compressing that data at the sensor buys back bandwidth, but only if the compressor is lossless, fast and small, because anything it discards is lost before target detection ever sees it.

This thesis compares data compression methods for automotive radar on real data and implements the most suitable of them in hardware. Eight methods were evaluated in MATLAB on 50 frames of the public ColoRadar dataset, recorded with a twelve-transmitter Texas Instruments cascade radar. The lossy quantisers reached compression ratios of up to 3.76 but missed up to 28.7 % of targets, and floating-point storage barely compressed at all. Two lossless predictive coders compressed by more than three: Doppler Redundancy Hybrid Encoding (DRHE), which predicts magnitude and phase with fixed filters, and a second-order linear predictive coder (LPC) whose coefficients are fitted to each frame. On a time-division MIMO radar both must predict each ramp from the previous ramp of the *same transmitter*, twelve ramps back; predicting from the adjacent ramp does worse than not predicting at all.

The compression ratio was broken down into its sources. Huffman coding of the 16-bit range-FFT data alone achieves 3.48; prediction adds a further 0.22–0.27; and a Huffman dictionary re-derived for the prediction errors adds 0.47 for DRHE and 0.71 for LPC, giving 4.22 and 4.40 overall, with every frame reconstructed exactly.

Both compressors and their decompressors were written in C++ and synthesised with Vitis HLS and Vivado, and the compressors were run on an AMD KCU116 (Kintex UltraScale+) board at 100 MHz. On the board all 50 frames were compressed byte-identically to the simulation, and every frame decompressed with zero error. DRHE compresses a frame in 0.61 ms (5.1 Gbit/s) and streams its output with 13 µs of latency. LPC takes 3.25 ms and, because its coefficients depend on the whole frame, cannot output anything until the whole frame has arrived: its last output word follows its last input word by 3.0 ms. Encoder and decoder together use 23 % (DRHE) and 25 % (LPC) of the chip's logic. LPC compresses about 4 % better; DRHE is recommended for a streaming sensor front end, because it needs no frame buffer and has a small fraction of LPC's latency.

Acknowledgements

I would like to thank A/Prof. Beena Ahmed for her guidance throughout this project, and Dr. Mostafa Shahin for his assessment and feedback. I am also indebted to Lukas Kiem, whose master's thesis provided the algorithmic baseline this work builds on; it was specified carefully enough to be reimplemented and tested exactly, which made much of this thesis possible.

Abbreviations

ADAS	Advanced Driver Assistance Systems
ADC	Analog-to-Digital Converter
AR	Autoregressive
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
BAQ	Block Adaptive Quantization
BRAM	Block Random Access Memory
CR	Compression Ratio
C-sim	C-Simulation (the HLS C++ run as software)
DDR	Double Data Rate (memory)
DMA	Direct Memory Access
DRHE	Doppler Redundancy Hybrid Encoding
DSP	Digital Signal Processing / DSP slice
EGE	Exponential-Golomb Encoding
FDR	False Discovery Rate
FF	Flip-Flop
FFT	Fast Fourier Transform
FMCW	Frequency-Modulated Continuous Wave
FN	False Negative
FP16	16-bit (half-precision) floating point
FPGA	Field-Programmable Gate Array
FX16	16-bit fixed-point (integer) representation
HLS	High-Level Synthesis
IF	Intermediate Frequency
IIR	Infinite Impulse Response
II	Initiation Interval
I/Q	In-phase / Quadrature
LPC	Linear Predictive Coding
LSB	Least Significant Bit
LUT	Look-Up Table
MIMO	Multiple-Input Multiple-Output
MSB	Most Significant Bit
RDVLE	Range-Dependent Variable-Length Encoding
RTL	Register-Transfer Level
RX	Receive / Receiver
SerDes	Serialiser/Deserialiser
SNR	Signal-to-Noise Ratio
SoC	System-on-Chip
TDM	Time-Division Multiplexing
TX	Transmit / Transmitter
UDRE	Uniform Dynamic-Range re-Encoding

WNS	Worst Negative Slack

Contents

Abstract i

Acknowledgements ii

Abbreviations iii

Contents v

List of Figures ix

List of Tables x

1 Introduction 1
- 1.1 The problem . . . 1
- 1.2 Motivation . . . 1
- 1.3 Why the compressor must be lossless . . . 2
- 1.4 Aim and objectives . . . 2
- 1.5 Contributions . . . 3
- 1.6 Thesis outline . . . 3

2 Background 5
- 2.1 How an FMCW radar measures range . . . 5
  - 2.1.1 The chirp . . . 5
  - 2.1.2 From beat frequency to a range profile . . . 5
  - 2.1.3 Slow time and Doppler . . . 6
- 2.2 Angle, MIMO and the virtual array . . . 7
  - 2.2.1 Why more antennas . . . 7
  - 2.2.2 MIMO: buying antennas cheaply . . . 7
  - 2.2.3 Time-division multiplexing, and why it matters here . . . 7
- 2.3 The radar data cube, and why it is large . . . 8
- 2.4 Lossless compression: how it actually works . . . 8
  - 2.4.1 The two categories . . . 8
  - 2.4.2 The bound: entropy . . . 9
  - 2.4.3 The standard two-stage structure . . . 9
  - 2.4.4 Open-loop versus closed-loop prediction . . . 9
- 2.5 The sensor and the dataset . . . 10
  - 2.5.1 AWR2243 and the MMWCAS-RF-EVM . . . 10
  - 2.5.2 ColoRadar . . . 10
- 2.6 Number representation and quantisation . . . 11
  - 2.6.1 Fixed point versus floating point . . . 11
  - 2.6.2 Quantisation error, and the FX16 step . . . 11
- 2.7 FPGAs and high-level synthesis . . . 11

2.7.1 What an FPGA is . . . 11
2.7.2 The four currencies: LUT, FF, BRAM, DSP . . . 12
2.7.3 Pipelining, initiation interval and $F_{\max}$ . . . 12
2.7.4 High-level synthesis . . . 13
2.7.5 AXI4-Stream . . . 13
2.8 Related work and the baseline . . . 13
2.8.1 Deterministic industrial schemes . . . 13
2.8.2 Block Adaptive Quantization . . . 13
2.8.3 Range-dependent variable-length encoding . . . 13
2.8.4 Predictive coding: DRHE and linear prediction . . . 13
2.8.5 Neural and spectral approaches . . . 14
2.9 The baseline: Kiem's thesis . . . 14
2.9.1 His problem and data . . . 14
2.9.2 His design-space exploration . . . 14
2.9.3 His final scheme and hardware . . . 14
2.9.4 What he left open . . . 15
2.10 Summary of what the later chapters assume . . . 15

3 Compression Methods 16
3.1 The eight candidate methods . . . 16
3.2 The shared entropy coder: turning residuals into bits . . . 17
3.2.1 What a residual is . . . 17
3.2.2 Why a re-expression can be worth anything . . . 17
3.2.3 The parsing problem . . . 18
3.2.4 The category S_4 . . . 18
3.2.5 The APPEND bits . . . 19
3.2.6 Cost, and the shape of the dictionary . . . 19
3.2.7 Worked example 1: one value, in full . . . 20
3.3 Predicting on a time-division MIMO radar . . . 21
3.3.1 Why not one model per transmitter? . . . 23
3.4 DRHE: Doppler Redundancy Hybrid Encoding . . . 24
3.4.1 Why predict in polar coordinates . . . 24
3.4.2 The two predictors . . . 24
3.4.3 The streaming property . . . 24
3.5 LPC: linear prediction with Yule–Walker coefficients . . . 25
3.5.1 Autoregressive modelling . . . 25
3.5.2 Stability . . . 25
3.5.3 Granularity and side information . . . 26
3.5.4 The cost: LPC is not causal . . . 26
3.5.5 Coefficient quantisation . . . 26
3.5.6 What this LPC is not . . . 26
3.6 The Huffman dictionary . . . 26
3.7 The two algorithms side by side . . . 27

4 Evaluation Method 28
4.1 Two stages . . . 28
4.2 The reference signal chain . . . 28
4.2.1 Loading . . . 28
4.2.2 Windowing . . . 28
4.2.3 Range FFT . . . 29
4.2.4 FX16 quantisation . . . 29
4.2.5 Detection and metrics . . . 29

4.3 Metrics, defined precisely . . . 30
4.3.1 Compression ratio . . . 30
4.3.2 Maximum absolute difference . . . 30
4.3.3 Detection metrics . . . 30
4.3.4 What the detection metrics actually measure here . . . 31
4.3.5 No-prediction baseline and order-0 entropy . . . 31
4.3.6 Timing . . . 31
4.4 The verification ladder . . . 31
4.5 Edge-case testing . . . 32
4.6 An independent implementation . . . 33
4.7 Reproducing the baseline . . . 33
4.8 Tools . . . 34

5 Hardware Implementation 35
5.1 Common interface and bit packing . . . 35
5.2 DRHE compressor . . . 35
5.2.1 State and parallelism . . . 35
5.2.2 Where the logic goes . . . 36
5.2.3 Initiation interval . . . 36
5.3 LPC compressor . . . 36
5.3.1 Three passes . . . 36
5.3.2 Reaching back n_{Tx} and $2n_{\mathrm{Tx}}$ ramps . . . 37
5.3.3 Where the cycles go . . . 37
5.3.4 Fixed-point choices . . . 37
5.3.5 An asymmetry that favours the decoder . . . 37
5.4 Exactly lossless on any input . . . 37
5.4.1 The one value the code cannot represent . . . 38
5.4.2 Wide-integer conversion in the coefficient solver . . . 38
5.5 Decompressors . . . 38
5.5.1 Why decoding is the harder half . . . 38
5.5.2 DRHE: a two-stage serial decoder . . . 39
5.5.3 LPC . . . 39
5.6 The board test system . . . 39
5.7 Building and timing closure . . . 40

6 Results 41
6.1 Stage 1: screening eight methods . . . 41
6.1.1 The cost of the quantisation step . . . 42
6.2 Stage 2: the prediction axis . . . 43
6.3 Where the compression comes from . . . 43
6.4 The re-derived dictionary . . . 45
6.5 Frame-to-frame variation . . . 46
6.6 Verification . . . 46
6.7 Speed and latency on the board . . . 47
6.8 Chip resources . . . 48
6.9 The two algorithms side by side . . . 49

7 Discussion 50
7.1 Which algorithm for a sensor front end? . . . 50
7.2 Scaling to larger frames . . . 51
7.3 What the compression ratio measures . . . 51
7.4 How far the dictionary and the lag generalise . . . 52

7.5 Hardware cost against the baseline design 52
7.6 Comparison with the baseline's results 53
7.7 Threats to validity 54

8 Conclusions and Future Work **55**
8.1 Summary 55
8.2 Findings 55
8.3 Future work 56

Bibliography **57**

Bibliography **57**

A The Huffman Dictionaries **59**

B Bitstream Layouts **61**
B.1 DRHE 61
B.2 LPC 61

C Reproducing the Results **62**
C.1 Source files 62
C.2 Commands 63

D Verification Record **64**

List of Figures

1.1 Raw versus compressed radar transport 2

2.1 FMCW chirp and beat frequency 6
2.2 TDM-MIMO ramp interleaving 8
2.3 Radar data cube and processing chain 8
2.4 Two-stage lossless compressor 9

3.1 The sixteen samples, plotted 22
3.2 DRHE block diagram 25

4.1 One input frame 29
4.2 The verification ladder 32

5.1 LPC three-pass architecture 36
5.2 The KCU116 test system 40

6.1 Compression against missed targets 42
6.2 Compression ratio, step by step 44
6.3 Bits needed per value 45
6.4 Compression ratio of each frame 46
6.5 Time per frame on the FPGA 47
6.6 FPGA resources 48

7.1 Where the bits go 51

List of Tables

2.1 ColoRadar cascade capture parameters used throughout this thesis. 10
2.2 Kiem's published results 15

3.1 Raw values against residuals 18
3.2 S4 category ranges 19
3.3 Category 3 enumerated 19
3.4 Sixteen samples traced through both predictors 21
3.5 Why the two lags differ 23
3.6 DRHE and LPC compared 27

4.1 Synthetic edge cases 33

5.1 The four board builds 40

6.1 Stage 1: the eight methods 41
6.2 Quantisation study 43
6.3 The prediction lag 43
6.4 Where the compression comes from 44
6.5 What the re-derived dictionary changes 45
6.6 Measured timing on the board 47
6.7 Resources per core 48
6.8 DRHE and LPC compared 49

7.1 Hardware against the baseline 52

A.1 The original Huffman dictionary 59
A.2 The re-derived dictionaries 60

D.1 Verification record 64

Chapter 1

Introduction

1.1 The problem

A modern high-resolution automotive radar generates data faster than the vehicle it is mounted on can conveniently move it. The cascaded radar used in this thesis produces about 3 MiB of raw digitised samples every frame, and it is one of several such sensors on a car. Getting that data to the central processor means either expensive shielded high-speed cabling or compressing it at the sensor first. Compression is the cheaper answer, but it has to keep up with the sensor, fit in a small amount of silicon, and — because this data feeds a system that decides whether to brake — lose nothing. This thesis compares the available compression methods on real radar data, selects the ones that meet those conditions, and builds and measures them on an FPGA.

1.2 Motivation

Radar earns its place in a vehicle's sensor suite because it works when the other sensors do not. Cameras fail in fog, glare and darkness; lidar is degraded by rain and airborne dust; radar sees through all of them [1, 2]. That robustness is why radar has been in production vehicles for two decades, initially for simple distance warning and now for adaptive cruise control, automatic emergency braking, blind-spot detection and lane-change assistance [3].

What has changed recently is resolution. Supporting SAE Level 2+ autonomy needs more than "something is 40 m ahead". It needs a map: how many objects, where, how fast, how wide, and what they probably are. Angular resolution is what separates "an object" from "a car and a pedestrian standing beside it", and angular resolution comes from having many antennas. A radar with N_t transmitters and N_r receivers can synthesise an effective array of $N_t \times N_r$ elements, so resolution improves roughly with the product. The data volume, unfortunately, scales the same way.

At the same time, vehicle electrical architectures have been consolidating. Where a car once had a hundred small electronic control units each doing one job, the industry has moved to domain controllers, then to *zonal* architectures where sensors report to a nearby gateway, and toward fully centralised designs where one high-performance system-on-chip does everything [4, 5, 6]. Centralising the computation has obvious advantages, but it means the raw sensor data has to travel.

Figure 1.1 shows the consequence. A Serialiser/Deserialiser (SerDes) link [7, 8] carries the aggregated output of several radar modules at 2.5 Gbit/s or more. Those data rates are achievable, but they require shielded cable, they cost money and weight, and they put a matching ingest burden on the SoC's memory system [9]. If instead the data is compressed at the radar module by a factor of 2.5 or better, the same information fits inside Gigabit

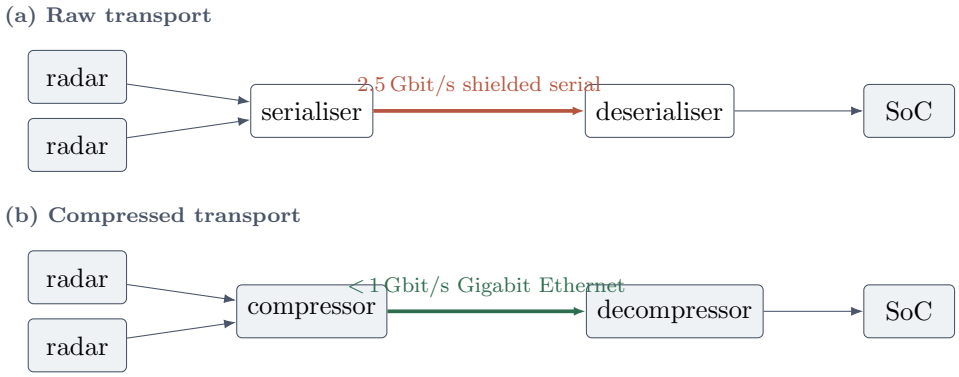


Figure 1.1: The economic argument for sensor-edge compression. Compressing by a factor of 2.5 or better at the radar module replaces an expensive shielded high-speed serial link with standard Gigabit Ethernet, without changing what the central processor receives.

Ethernet [10] — a cheap, standard, unshielded interconnect. That single substitution is the economic argument for this entire body of work.

1.3 Why the compressor must be lossless

It is tempting to relax the requirement. Lossy compression reaches much higher ratios, and radar data is noisy anyway, so surely a little extra noise is harmless?

It is not, and the reason is specific rather than philosophical. The compressor sits *upstream* of detection. Whatever it discards is discarded before the detector ever sees it, and the detector's job is to find weak returns close to the noise floor — a pedestrian at range, a motorcycle between two trucks. Lossy compression raises the noise floor precisely where the margin is thinnest. Chapter 6 quantifies this on real data: Block Adaptive Quantization reaches the same compression ratio as the lossless coders studied here but misses nearly three times as many real detections and reports nearly nine times as many false ones.

Losslessness also makes verification unambiguous. The pass criterion is that every reconstructed sample equals its original exactly. There is no tolerance to argue about and no distortion metric to trade off: a single differing bit anywhere in the 157 million input bits of the test set is a failure, and the test reports it as one. Every design in this thesis is checked against that criterion at every level, from MATLAB to the FPGA board.

1.4 Aim and objectives

The starting point is the master's thesis of Lukas Kiem [11], which evaluated a range of radar compression schemes on recordings from a single-transmitter Infineon radar, selected Doppler Redundancy Hybrid Encoding (DRHE) as the best of them, and implemented its compressor on a Zynq UltraScale+ device (compression ratio 3.17 in hardware). Section 2.9 describes that work. It left four questions open, which became the objectives of this thesis.

Aim. Compare compression methods side by side on real, public, multi-transmitter radar data; select the best *lossless* methods; and implement both their encoders and their decoders on an FPGA, measured on a real board.

1. **Validate on public MIMO data.** Kiem's results come from private recordings of a single-transmitter radar. This thesis uses 50 consecutive frames of the public ColoRadar

dataset [12], recorded with a Texas Instruments twelve-transmitter, sixteen-receiver cascade, for every result from simulation to hardware.

2. **Examine the quantisation step.** Kiem identified the conversion to 16-bit fixed point (FX16) as the source of his detection errors. This thesis measures what that step costs in detection, and whether floating-point storage is a usable alternative.
3. **Broaden the comparison.** Eight methods are compared on the same data with the same metrics: two reference formats, three lossy quantisers, DRHE on integer and on floating-point data, and a linear predictive coder (LPC) designed for this thesis as a controlled comparison with DRHE.
4. **Implement and measure the complete pipeline.** Kiem implemented compression only and measured it on his own system. This thesis implements compression *and* decompression for both selected algorithms, verifies them bit-exactly at every level of abstraction, and measures throughput, latency and resources on an AMD KCU116 board.

1.5 Contributions

1. **A comparison of eight compression methods on real cascade radar data** (Section 6.1). Only the two predictive lossless coders, DRHE (3.75) and LPC (3.70), compress by more than three without losing detections; the lossy quantisers reach up to 3.76 but miss 10 % to 29 % of targets, and floating-point storage compresses by only 1.04.
2. **The correct prediction axis for time-division MIMO radar** (Section 3.3). On a TDM cascade consecutive ramps come from different transmitters, so slow-time prediction must use the previous ramp of the same transmitter, n_{Tx} ramps back. Predicting from the adjacent ramp, as single-transmitter descriptions of these algorithms do, compresses worse than not predicting at all.
3. **A breakdown of where the compression comes from** (Sections 6.3 and 7.3). Huffman coding alone achieves 3.48 on this data — mostly from unused fixed-point headroom and the sparsity of a range-FFT frame — prediction adds 6 % to 8 %, and the Huffman dictionary decides the rest.
4. **Huffman dictionaries matched to the prediction errors** (Section 6.4). Re-derived from five frames, they raise DRHE to 4.22 and LPC to 4.40, hold on the 45 frames they were not trained on, and reverse the ranking of the two algorithms.
5. **Complete, verified hardware for both algorithms** (Chapters 5 and 6). Encoders and decoders were implemented in Vitis HLS and Vivado and run in a board test system on an AMD KCU116. Compressed output is byte-identical to the simulation on all 50 frames, reconstruction is exact, and throughput, latency and resources are measured.
6. **A comparison with the baseline** (Section 7.6), including a reconstruction of Kiem's simulation from his text, which reproduces his results and explains how they relate to the ones here.

1.6 Thesis outline

Chapter 2 gives the necessary background: how an FMCW radar measures range and velocity, how MIMO and time-division multiplexing produce a virtual array, why the data is large, how lossless compression works, the sensor and dataset, number representation, FPGAs and high-level synthesis, and related work including Kiem's thesis. Chapter 3 describes the eight compression methods, the entropy coder shared by the two predictive ones, the choice of prediction axis, the two predictors in detail, and the Huffman dictionary. Chapter 4 sets

out how they are evaluated: the two-stage evaluation, the signal chain, the metrics and the verification ladder from MATLAB to the board. Chapter 5 describes the hardware architectures of the encoders and decoders and the board test system. Chapter 6 reports the results, Chapter 7 discusses what they mean and how they compare with the baseline, and Chapter 8 concludes and proposes future work.

Chapter 2

Background

This chapter assumes no prior knowledge of radar. It builds up, in order, what the radar measures, how it measures it, what the resulting data looks like, why there is so much of it, how lossless compression works in general, and what an FPGA can do about it. Each section ends with the specific fact that the later chapters depend on.

2.1 How an FMCW radar measures range

2.1.1 The chirp

Automotive radar is almost universally *frequency-modulated continuous wave* (FMCW). Instead of emitting a short pulse and timing the echo — which would demand impractically fast electronics for the distances involved — an FMCW radar transmits continuously, sweeping its frequency linearly upward over a short interval. One such sweep is called a *chirp*. The sensor used here sweeps 1.798 GHz of bandwidth around a 77 GHz centre frequency while its echo is sampled, and starts a new chirp every 78.5 µs.

When the chirp reflects off an object at range R, the echo arrives delayed by the round-trip time

$$\tau = \frac{2R}{c}, \tag{2.1}$$

where c is the speed of light. Because the transmitter has kept sweeping in the meantime, the echo's frequency at any instant is lower than what is currently being transmitted, by an amount proportional to the delay. If the chirp sweeps B hertz in T_c seconds, the sweep slope is $S = B/T_c$ and the frequency difference is

$$f_b = S\,\tau = \frac{2SR}{c}. \tag{2.2}$$

A mixer multiplies the transmitted and received signals, producing a low-frequency *beat* or intermediate-frequency signal at exactly f_b. That signal is low-pass filtered and digitised by an analog-to-digital converter (ADC) at a modest rate — 8 MHz here, rather than the tens of gigahertz the raw RF would need. This is the central trick of FMCW: it converts a timing measurement that would need picosecond resolution into a frequency measurement that an ordinary ADC can make.

2.1.2 From beat frequency to a range profile

Measuring f_b is a spectral estimation problem, so the natural tool is the Fourier transform. Taking an N_s-point FFT across the N_s ADC samples of a single chirp produces a *range profile*: bin n of the FFT corresponds, through Equation (2.2), to range

$$R_n = n \cdot \frac{c}{2B}, \tag{2.3}$$

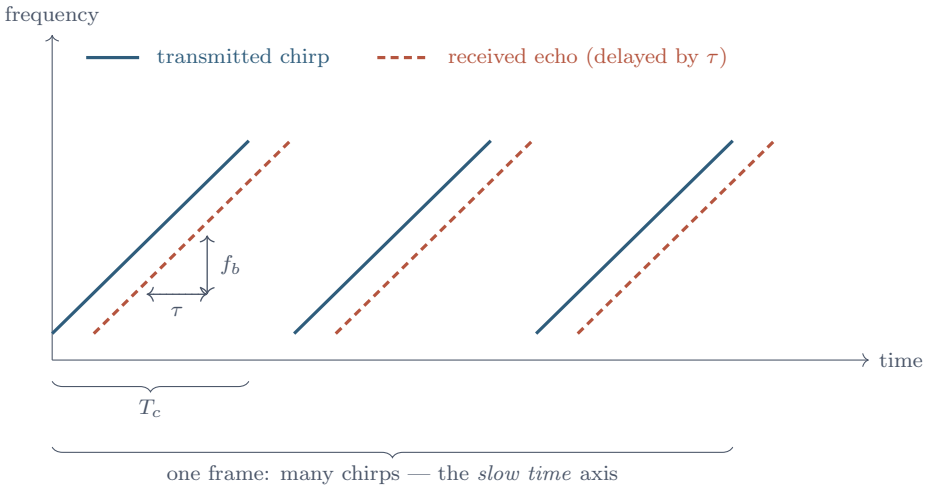


Figure 2.1: FMCW operation. Each chirp sweeps linearly in frequency. The echo is a delayed copy, so at any instant the transmitted and received frequencies differ by $f_b = S\tau$, proportional to target range. A frame consists of many such chirps in succession; the axis running across chirps is called *slow time* and is where Doppler information lives.

so the range resolution is

$$\Delta R = \frac{c}{2B} \approx 8.3\,\text{cm} \tag{2.4}$$

for the 1.798 GHz bandwidth used here. A peak in the range profile is an object. This first transform is called the **Range FFT**, and its output is the data this thesis compresses.

2.1.3 Slow time and Doppler

A single chirp gives range but not velocity. To get velocity, the radar transmits many chirps in rapid succession — a *frame* — and looks at how the *phase* of a given range bin evolves from chirp to chirp.

The reasoning is worth spelling out, because the whole of DRHE rests on it. A target at range R produces a range-FFT value whose phase is $\theta = -4\pi R/\lambda$, where λ is the wavelength. Over the chirp-to-chirp interval T, a target moving at radial velocity v changes range by vT, so its phase changes by

$$\Delta\theta = -\frac{4\pi v T}{\lambda}. \tag{2.5}$$

At 77 GHz, $\lambda \approx 3.9$ mm, so even a very slow target produces a measurable phase step — which is exactly why radar is good at velocity. Critically for compression, $\Delta\theta$ is *constant* for a target at constant velocity. The phase of a given range bin therefore advances linearly across chirps, and its magnitude barely changes at all. That is a highly predictable sequence, and predictable sequences compress.

The axis running across chirps within a frame is called **slow time**, as opposed to the *fast time* axis of samples within a chirp. Taking a second FFT along slow time resolves $\Delta\theta$ into velocity: this is the **Doppler FFT**.

Carry forward: slow time is where the redundancy is. A target at constant velocity produces a range-bin sequence whose magnitude is nearly constant and whose phase advances by a constant increment. Both DRHE and LPC exist to exploit exactly this.

2.2 Angle, MIMO and the virtual array

2.2.1 Why more antennas

Range and velocity are not enough to drive a car. The radar also has to know *where* a target is in angle, and how well it can separate two targets at the same range and velocity but different bearings.

Angle is measured by comparing the phase of the same echo at physically separated receive antennas. A wavefront arriving at angle ϕ reaches adjacent antennas, spaced d apart, with a path difference $d \sin \phi$, hence a phase difference

$$\Delta\psi = \frac{2\pi d \sin \phi}{\lambda}. \tag{2.6}$$

An FFT across the receive-antenna axis — the **Angle FFT** — turns those phase differences into a bearing estimate. The angular resolution improves with the physical size of the array, which in practice means with the number of elements.

2.2.2 MIMO: buying antennas cheaply

Building a large physical receive array is expensive. Multiple-Input Multiple-Output (MIMO) radar gets the same effect more cheaply. If N_t transmitters and N_r receivers are arranged suitably, and the signal from each transmitter can be separated at each receiver, then each of the $N_t \times N_r$ transmit–receive pairs behaves like an element of a *virtual array*. The sensor used here has $N_t = 12$ and $N_r = 16$, giving 192 virtual elements from 28 physical antennas.

2.2.3 Time-division multiplexing, and why it matters here

The separability requirement has to be met somehow. The simplest method, and the one this sensor uses, is *time-division multiplexing* (TDM): the transmitters fire one at a time, in a fixed rotation, so whatever a receiver hears during transmitter t's slot must have come from transmitter t.

TDM is simple and robust, and it has one consequence that matters throughout this thesis. Inside every repetition of the frame — every "chirp loop" — all N_t transmitters fire in sequence before the loop repeats. When the resulting data is laid out as a two-dimensional array of (ramp, range bin), which is how a streaming compressor sees it, the ramp index is

$$m = q\,N_t + t, \qquad q \in [0, N_c), \quad t \in [0, N_t), \tag{2.7}$$

where q indexes the chirp loop and t the transmitter. The transmitter index varies *fastest*.

This is the correct physical time order. It is also, as Figure 2.2 makes visual, a trap. Ramp m and ramp $m-1$ are different physical transmitters $N_t - 1$ times out of N_t. They are separated in *space* by the array baseline, not in *time* by a pulse interval, and the phase relationship between them is governed by Equation (2.6) — the target's bearing — not by Equation (2.5) — the target's velocity. The genuine slow-time neighbour of ramp m, the one whose phase differs by a Doppler step, is ramp $m - N_t$. From Chapter 3 on, this distance is written n_{Tx}; on this sensor $n_{\mathrm{Tx}} = N_t = 12$.

Carry forward: on a TDM-MIMO radar the ramp axis is transmitter-interleaved. A slow-time predictor that assumes consecutive ramps are consecutive *in time on the same antenna* — as DRHE does in its single-transmitter form, and as a plain lag-1/lag-2 linear predictor does — is predicting across the wrong axis. Section 3.3 sets the prediction lag to n_{Tx} for this reason, and Chapter 6 measures the difference it makes.

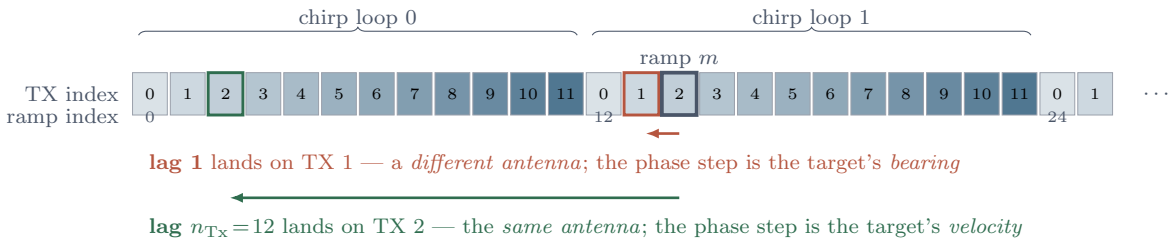


Figure 2.2: Why "the previous ramp" is the wrong neighbour on a TDM-MIMO cascade. Each cell is one ramp, labelled with the transmitter that produced it; the twelve transmitters rotate inside every chirp loop. Predicting ramp m from ramp $m-1$ (red) crosses a transmitter boundary eleven times out of twelve, so the two ramps differ by the target's array steering phase. Predicting from ramp $m-n_{\mathrm{Tx}}$ (green) stays on the same physical antenna and therefore on the genuine Doppler axis.

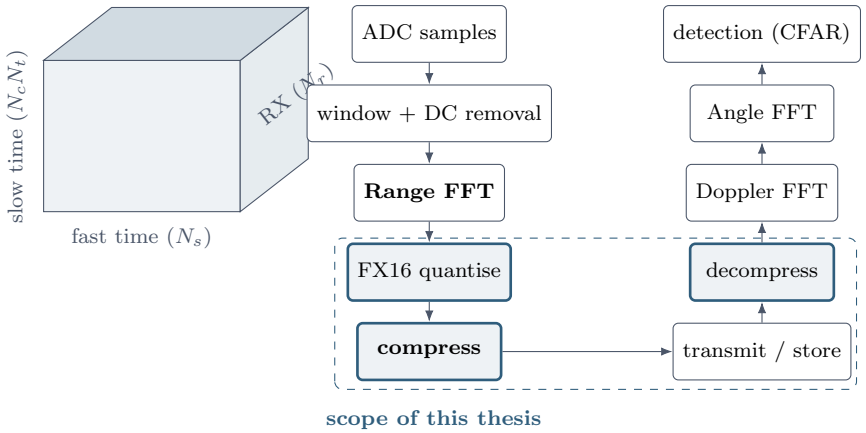


Figure 2.3: The radar data cube and the processing chain. The compressor sits immediately after range-FFT and fixed-point quantisation, before the data leaves the sensor. Everything downstream — Doppler FFT, angle FFT, detection — sees the reconstructed data, which for a lossless compressor is bit-identical to the original.

2.3 The radar data cube, and why it is large

Putting the three axes together gives the *radar data cube*: fast time (samples within a chirp) × slow time (ramps) × receive channel. Three FFTs are applied along the three axes in turn, yielding range, velocity and angle. Figure 2.3 shows the arrangement and where in the chain this thesis intervenes.

The volume follows directly. For the sensor used here, one frame is

$$N_s \times N_c \times N_r \times N_t = 256 \times 16 \times 16 \times 12 = 786{,}432 \tag{2.8}$$

complex samples. Each is two 16-bit integers, so a frame is 3,145,728 B — exactly 3 MiB. At 20 frames per second, a common automotive rate, that is about half a gigabit per second from one sensor, and a vehicle carries several.

2.4 Lossless compression: how it actually works

2.4.1 The two categories

Compression divides into *lossless*, where the decompressed data is bit-identical to the input, and *lossy*, where some information is permanently discarded in exchange for a smaller payload [14]. Lossy compression can reach far higher ratios; lossless compression is bounded by the

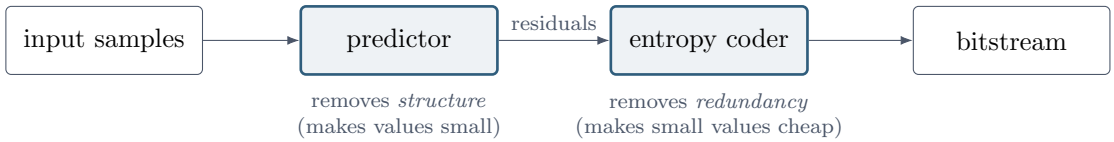


Figure 2.4: The standard structure of a lossless compressor. The two stages do different jobs, and each can succeed or fail independently of the other.

information content of the data itself. Section 1.3 explained why this application needs the lossless kind.

2.4.2 The bound: entropy

There is a hard limit on lossless compression, and knowing it is what makes the results in Chapter 6 interpretable. If a source emits symbols x with probabilities $p(x)$, its *order-0 entropy* is

$$H = -\sum_{x} p(x) \log_2 p(x) \quad \text{bits per symbol,} \tag{2.9}$$

and no code that treats symbols independently can average fewer bits than H [14]. This gives a concrete yardstick: measure H for the data, compare it against what the coder actually achieves, and the gap is the coder's inefficiency. Section 6.3 uses exactly this to separate what the predictor contributes from what the Huffman dictionary contributes.

2.4.3 The standard two-stage structure

Almost every practical lossless compressor has the same shape (Figure 2.4): a *decorrelation* stage followed by an *entropy coding* stage.

The **decorrelation** stage exploits structure. If consecutive samples are similar, predict each one from its neighbours and transmit only the *residual*, the difference between prediction and truth. When the prediction is good, residuals cluster tightly around zero even though the original samples spanned the full dynamic range. Nothing is lost: given the residual and the same prediction rule, the decoder reconstructs the original exactly.

The **entropy coding** stage exploits the resulting skew in the distribution. Values that occur often get short codewords; values that occur rarely get long ones. Huffman coding [15] is the classic construction and produces the optimal prefix-free code for a known symbol distribution.

Crucially, the two stages are independent. A predictor that fails — that produces residuals no more skewed than its input — does not break the coder; the coder simply carries on compressing whatever it is handed, and the compression ratio can still look respectable. Detecting this requires deliberately measuring what the coder achieves *without* the predictor. This thesis reports that no-prediction baseline alongside every predictive result.

2.4.4 Open-loop versus closed-loop prediction

One subtlety in predictive coding matters enough to state now, because the hardware of Chapter 5 is built around it (Section 5.4).

The encoder predicts from past samples. So does the decoder — but the decoder does not have the past *original* samples, only the past *reconstructed* ones. In *open-loop* prediction the encoder uses the originals; in *closed-loop* prediction (classically, differential pulse-code modulation, DPCM) the encoder reconstructs each sample exactly as the decoder will, and predicts from that.

Table 2.1: ColoRadar cascade capture parameters used throughout this thesis.

Parameter	Symbol	Value
ADC samples per chirp	N_s	256
Chirp loops per frame	N_c	16
Receive channels	N_r	16
Transmit channels (TDM)	N_t	12
Ramps per frame	$N_c N_t$	192
Fast-time sample rate	f_s	8 MHz
Chirp bandwidth	B	1.798 GHz
Ramp repetition interval	T_r	78.5 µs
Centre frequency	f_c	77 GHz
Range resolution	ΔR	8.34 cm
Frame size (raw)	—	3,145,728 B

When compression is perfectly lossless the two are identical, because reconstruction equals the original. The moment anything — a saturation, a rounding, a clamp — makes reconstruction differ from the original by even one bit, an open-loop encoder and its decoder begin predicting from different histories and *diverge*. With an IIR predictor, as in DRHE, that divergence is then carried forward through every subsequent ramp. Closing the loop confines the error to the affected sample.

Carry forward: a compression ratio is the product of two independent effects, and a healthy-looking ratio does not imply a working predictor. Always measure the entropy coder alone as a baseline.

2.5 The sensor and the dataset

2.5.1 AWR2243 and the MMWCAS-RF-EVM

The radar front end assumed throughout is the Texas Instruments AWR2243, a 76–81 GHz FMCW transceiver with three transmit and four receive channels [13]. Importantly for this work, the AWR2243 performs no signal processing of its own: it transmits, receives, mixes and digitises, then streams raw ADC samples out over a MIPI CSI-2 interface. It therefore exposes exactly the payload that creates the transport bottleneck.

Four AWR2243 devices are cascaded on the MMWCAS-RF-EVM evaluation module, giving 12 transmit and 16 receive channels on one synchronised board — the 192-element virtual array of Section 2.2.

2.5.2 ColoRadar

Rather than synthetic stimulus, this thesis uses real captures from the public ColoRadar dataset [12], which includes cascade recordings from exactly this module across indoor, outdoor and subterranean environments. Table 2.1 lists the relevant parameters. Fifty consecutive frames were used throughout; that is 39,321,600 complex ADC samples, or 150 MiB of raw data, which is enough to make a claim of losslessness meaningful.

Each complex sample is stored as two contiguous `int16` values. The index of the sample at (fast-time s, chirp loop q, receiver r, transmitter t) in the file is

$$i = 2\big(s + N_s(q + N_c(r + tN_r))\big), \tag{2.10}$$

so the layout from slowest to fastest varying is $[t, r, q, s]$. Reshaping this into the (fast time, ramp, receiver) cube the processing chain expects is what produces the transmitter-interleaved ramp axis of Equation (2.7).

2.6 Number representation and quantisation

2.6.1 Fixed point versus floating point

Hardware has to represent numbers somehow, and the choice is consequential.

Fixed-point representation allocates a constant number of bits to the integer and fractional parts, commonly written $Qm.n$. Because the binary point never moves, addition, subtraction and multiplication are ordinary integer operations: cheap, single-cycle, and small in silicon [16]. The price is a fixed dynamic range. Values too large wrap around; values too small round to zero.

Floating-point representation stores a sign, exponent and mantissa, so the binary point moves with the magnitude [17]. This buys enormous dynamic range but costs a great deal: every operation requires exponent alignment, normalisation and special-case handling, which needs a dedicated floating-point unit and takes multiple clock cycles. Section 5.2 shows what this costs in the DRHE design: the floating-point operators account for about two thirds of its LUTs and all of its DSP slices.

2.6.2 Quantisation error, and the FX16 step

The range-FFT output is naturally a complex floating-point quantity. Before it can be compressed by a fixed-point pipeline it has to be quantised to 16-bit integers — the **FX16** step. This is a scaling followed by rounding:

$$\sigma = \frac{32767}{A_{\max}\, 2^{\text{shift}} \sum_k w_k \,/\, (2N_s)}, \qquad x_{\text{FX16}} = \text{round}(\sigma \cdot x), \tag{2.11}$$

where $A_{\max}$ is the full-scale ADC amplitude and w is the window applied along fast time.

Two things about Equation (2.11) matter later. First, the rounding introduces an error bounded by half a least-significant bit, which acts as a wideband noise floor [18]; this is the dominant source of detection degradation in the whole chain, and Chapter 6 separates it cleanly from the cost of compression. Second, and more consequentially for this thesis, the scale factor is *fixed* — it is calibrated from a theoretical full-scale ADC sinusoid, not from the data. If the actual capture sits well below full scale, the top bits of every resulting `int16` are only copies of the sign bit. On ColoRadar the largest FX16 code is 437, so fewer than ten of the sixteen bits ever carry information; Section 7.3 shows how much of the compression ratio this accounts for.

Carry forward: the FX16 scale factor is fixed and calibrated to full scale. Whether the capture actually fills the container is a property of the sensor's gain staging, and it silently inflates any compression ratio measured downstream.

2.7 FPGAs and high-level synthesis

2.7.1 What an FPGA is

A field-programmable gate array is a chip full of small configurable logic blocks and configurable wiring between them. Rather than executing instructions, an FPGA *becomes* the circuit you describe. The practical consequences for this application are three:

- **Parallelism.** A processor handles four receive channels one after another. An FPGA can instantiate four copies of the datapath and do all four in the same clock cycle. Every design in this thesis does exactly that.
- **Deterministic latency.** There is no cache, no operating system, no interrupt jitter. The DRHE compressor of this thesis takes 61,472 or 61,473 cycles on every one of the 50 test frames, which is what a safety-critical pipeline needs.
- **Reconfigurability.** The design can be changed after fabrication, which is what makes an FPGA the right platform for evaluating designs that are still being compared.

In production, a design proven on an FPGA is normally migrated into an ASIC or into a fixed-function accelerator block inside a larger SoC, for cost and power reasons [19, 21]. The FPGA here is a proof of concept, not a proposed product.

2.7.2 The four currencies: LUT, FF, BRAM, DSP

FPGA resource usage is reported in four units, and it is worth knowing what each is because the two algorithms in this thesis trade them against each other in an interesting way.

LUT
: *Look-up table.* A small memory that implements an arbitrary Boolean function of a few inputs. LUTs are the raw material of combinational logic. Complicated arithmetic — square roots, arctangents — costs a lot of LUTs.

FF *Flip-flop.* A one-bit register. Flip-flops hold values between clock cycles; deep pipelines need many of them.

BRAM
: *Block RAM.* Dedicated on-chip memory, here counted in 36 kbit blocks. Arrays that are too large to live in flip-flops go here. BRAMs typically have two access ports, which becomes important below.

DSP
: *DSP slice.* A hardened multiply–accumulate unit. Using a DSP for a multiply is far cheaper than building one from LUTs.

The target device throughout is an AMD Kintex UltraScale+ `xcku5p-ffvb676-2-e`, which provides 216,960 LUTs, 433,920 flip-flops, 480 BRAMs of 36 kbit and 1,824 DSP slices.

2.7.3 Pipelining, initiation interval and $F_{\max}$

Three timing concepts recur throughout the results.

Pipelining splits a long computation into stages separated by registers, so several inputs can be in flight at once. The *depth* of a pipeline is how many cycles a single input takes to traverse it.

Initiation interval (II) is how many cycles must elapse between accepting one input and the next. $\mathrm{II} = 1$ means a new sample every cycle, which is the goal. $\mathrm{II} = 2$ halves throughput. II is limited by the tightest resource constraint in the loop body — very often, as Chapter 5 shows, by the number of access ports on a block RAM.

$F_{\max}$ is the highest clock frequency at which the longest combinational path still settles within one period. HLS estimates it before the circuit exists; the figure that counts is Vivado's, after routing. The two can disagree badly: both compressors in this thesis were estimated by HLS below 100 MHz, and both close above it once placed and routed (Section 5.7).

Latency, throughput and $F_{\max}$ interact: a frame of K beats at initiation interval II and clock f takes roughly $K \cdot \mathrm{II}/f$ seconds, *plus* the pipeline depth once per invocation. That last term is often small, but not always: the LPC compressor of Section 5.3 invokes two short

loops 512 times per frame, and their pipeline fill costs about 65,000 cycles that a trip-count model misses entirely.

2.7.4 High-level synthesis

Traditionally FPGA designs are written in a hardware description language such as VHDL or Verilog, at the level of registers and wires. *High-level synthesis* (HLS) instead takes annotated C++ and generates the RTL automatically. Loops become state machines; arrays become memories; pragmas tell the tool to pipeline a loop, partition an array, or expose a port as an AXI4-Stream interface.

HLS is used throughout this thesis for a specific reason beyond convenience: because the design source is C++, the *same source* can be compiled and run as ordinary software against the MATLAB reference, before any hardware is generated. That makes the verification ladder of Section 4.4 possible: all 50 frames can be checked bit for bit in minutes rather than in hours of RTL simulation.

2.7.5 AXI4-Stream

The interface used by both compressors is AXI4-Stream: a simple valid/ready handshake carrying a fixed-width data word per beat. The designs here accept 128 bits per beat — four receive channels, each a 16-bit real and a 16-bit imaginary part — and emit 256 bits per beat of compressed bitstream. A useful consequence for benchmarking is that "128 bits per cycle" is the theoretical maximum input rate, and therefore what a perfectly efficient design at $\mathrm{II} = 1$ would achieve. Section 7.5 uses this to interpret a published throughput figure.

2.8 Related work and the baseline

2.8.1 Deterministic industrial schemes

Because automotive safety systems demand bounded latency, commercial radar processors use lightweight deterministic compressors. Texas Instruments uses Exponential-Golomb Encoding [20]; NXP's S32R family uses block floating-point formats such as CP4D, in which several mantissas share one exponent [21]. These are described as "near lossless" — CP4D introduces under 0.25 dB of signal loss — and reach ratios of roughly 2 to 4.

2.8.2 Block Adaptive Quantization

BAQ originates in synthetic aperture radar [22]. It divides the data into blocks, estimates each block's statistics, and adapts the quantiser step size accordingly, so weak blocks are quantised finely and strong blocks coarsely. It is lossy. It appears in this thesis as a comparison point, and Chapter 6 shows what its losses cost in detection terms.

2.8.3 Range-dependent variable-length encoding

Received power falls with range, so far range bins hold smaller values than near ones. Range-dependent variable-length encoding (RDVLE) [26] exploits this by spending fewer bits on far bins; a second stage, uniform dynamic-range re-encoding (UDRE), re-quantises blocks of bins to a fixed width. Both are lossy, and both are among the eight methods compared here.

2.8.4 Predictive coding: DRHE and linear prediction

Doppler Redundancy Hybrid Encoding [26, 25] predicts each range-FFT sample along slow time from earlier ramps of the same range bin, using two infinite-impulse-response filters in

polar form, and entropy-codes the prediction error. Linear predictive coding is the general form of the same idea: each sample is predicted as a weighted sum of earlier ones, with weights fitted to the data. Meucci and Mancuso [27] apply it to raw ADC samples along fast time with a Huffman code derived from the data. Chapter 3 specifies both predictors as used here.

2.8.5 Neural and spectral approaches

Autoencoder-based compressors reach high ratios but are expensive to deploy at the sensor [23]. AdaRadar [24] avoids this by keeping the edge side simple — a discrete cosine transform and spectral pruning — and placing the neural decoder at the SoC, with a feedback loop that adapts the rate to downstream perception performance. These are outside the scope of this thesis, which concerns itself with lossless schemes, but they define the upper end of the ratio spectrum.

2.9 The baseline: Kiem's thesis

The direct predecessor of this work is the master's thesis of Lukas Kiem [11], carried out at Infineon. Because most comparisons in this thesis are made against it, this section sets out what he did, on what data, and what he left open.

2.9.1 His problem and data

Kiem defined the transport problem with a use case: one Infineon CTRX8191F radar MMIC, four receive channels, 12-bit samples at 50 MS/s, giving 2.4 Gbit/s of raw data, in frames of 1024 ramps of 1024 samples that take about 21 ms to acquire. Carrying it over one Gigabit Ethernet link therefore needs a compression ratio of 2.4 (2.7 with protocol headers). The sensor cycles at 20 Hz, and he allowed compression 15 ms of the 50 ms cycle.

His radar used a *single transmitter* and delivered *real* IF samples. Consecutive ramps are therefore consecutive observations by the same antenna, so the transmitter interleaving of Section 2.2.3 never arises for him. Every compression result he publishes was measured on private recordings of this radar (512 samples × 512 ramps, four channels, 14-bit data); a synthetic scene appears in his thesis only as an illustration.

2.9.2 His design-space exploration

Kiem compared FP16 and FX16 references, Infineon's common-exponent formats CMPRA and CMPRB, Texas Instruments' Exponential-Golomb variants EGE4–EGE10, and DRHE, using a MATLAB chain of pre-processing, range FFT, compression, Doppler FFT, non-coherent integration and peak detection — the same chain this thesis adopts (Section 4.2). DRHE reached a compression ratio of 3.25 with exactly FX16's detection metrics, while the lossy formats either stayed at or below the required ratio or lost detections. He did not implement linear predictive coding, judging the raw-ADC method of [27] less promising than DRHE.

2.9.3 His final scheme and hardware

For hardware he simplified DRHE in two ways: he dropped its run-length code, which cost almost nothing (3.25 to 3.24), and he fixed the Huffman dictionary, deriving it once from one of his recordings so that it need not be computed or transmitted per frame. That dictionary is used here as the *original dictionary* (Appendix A).

He implemented the compressor with Vitis HLS and Vivado 2022.2 on a Zynq UltraScale+ `XCZU3CG`, in fixed-point arithmetic at II = 1 and 100 MHz, inside a system that streamed data through an FFT core and the compressor by DMA. Table 2.2 lists his results.

Table 2.2: Kiem's published results [11, Sections 3.5 and 5.2].

Quantity	Value
Compression ratio, MATLAB / hardware	3.25 / 3.17
Compressor IP: LUT / FF / BRAM (36 Kb) / DSP	45,892 / 9,243 / 10 / 44
Clock, initiation interval	100 MHz, 1
System latency (last DMA read to last DMA write)	79.4 µs
Input throughput (theoretical)*	11.92 Gibit/s

*Gibit = 2^{30} bits; 11.92 Gibit/s is 12.8 Gbit/s.

2.9.4 What he left open

He concluded that DRHE met every requirement of his use case. His suggestions for future work included investigating floating-point arithmetic for the prediction, a bit-accurate decompressor, and tests with a real sensor and other configurations. Four gaps follow, and they are the objectives of Section 1.4: his results come from a private single-transmitter dataset; the detection loss of the FX16 step was identified but not resolved; only one predictive coder was evaluated; and no decompressor was built, so the hardware could not be checked bit for bit.

2.10 Summary of what the later chapters assume

1. The data to be compressed is the *range-FFT output*, quantised to 16-bit integers, arranged as (ramp × range bin × receiver).
2. The redundancy worth exploiting lives along *slow time*: a target at constant velocity gives nearly constant magnitude and a constant phase increment.
3. On this sensor the ramp axis is *transmitter-interleaved*, so the slow-time neighbour of ramp m is ramp $m - N_t$, not ramp $m - 1$.
4. Lossless compression is required, so a method is judged first on whether it loses detections and only then on its compression ratio.
5. A compression ratio is the product of a decorrelation gain and an entropy-coding gain, and the two must be measured separately.
6. The FX16 scale factor is fixed to full scale, so container occupancy is a confounding variable in any measured ratio.
7. FPGA cost is measured in LUTs, flip-flops, BRAMs and DSPs, and timing in initiation interval, pipeline depth and $F_{\max}$.

Chapter 3

Compression Methods

This chapter describes the methods compared in this thesis. It starts with the eight candidates and what each one does, then specifies in detail the two that were taken to hardware: the entropy coder they share, the axis along which they predict, the two predictors themselves, and the Huffman dictionary that turns their prediction errors into bits.

3.1 The eight candidate methods

Every method operates on the same input: the range-FFT output of each frame, quantised to 16-bit integers by the FX16 step of Section 2.6.2, except where a method replaces that step. All eight were implemented in the same MATLAB chain and scored with the same metrics (Chapter 4).

FP16, FX16
: The two reference formats, as in Kiem's thesis: 16-bit floating point, and the 16-bit integer codes of the FX16 step. Neither compresses; FX16 is the input to every integer method below, so its detection losses are the floor that any of them can reach.

BAQ
: Block adaptive quantisation, implemented in block-floating-point form: each block of 16 range bins shares an 8-bit exponent chosen from the block maximum, and each real and imaginary part keeps a 4-bit mantissa, so the ratio is fixed at $32/(8 + 8/16) = 3.76$. It stands in for the block-floating-point family (NXP's CP4/CP8 and Infineon's CMPRA/CMPRB are of the same kind) and for BAQ proper [22]. Lossy.

RDVLE
: Range-dependent variable-length encoding [26]: fewer bits for far range bins, following the fall of received power with range, here capped at three bits removed. Lossy.

RDVLE+UDRE
: RDVLE followed by uniform dynamic-range re-encoding of blocks of 16 bins at six bits [26]. Lossy.

DRHE
: Doppler Redundancy Hybrid Encoding, as specified by Kiem (Section 3.4): slow-time prediction in polar form, followed by the entropy coder of Section 3.2. Lossless with respect to the FX16 codes.

DRHE fp16
: The same predictor applied to FP16 values instead of FX16 codes, to test whether floating-point storage — the remedy Kiem suggested for quantisation error — can be compressed.

LPC
: A second-order linear predictive coder designed for this thesis (Section 3.5). It predicts

along the same axis as DRHE and uses the same entropy coder, so that only the predictor differs. Lossless with respect to the FX16 codes.

NXP's formats were not implemented separately, since their exact bit layouts are proprietary and the block-floating-point quantiser represents the family. Neural and spectral methods such as AdaRadar [24] were left out of scope: they are lossy, and their decoders run on the central processor.

The rest of this chapter concerns DRHE and LPC, which Section 6.1 shows are the only two methods that compress by more than three without losing detections.

3.2 The shared entropy coder: turning residuals into bits

This section is deliberately detailed. The entropy coder is where the compression ratio is physically realised — it is the only part of either design that actually emits bits — so understanding exactly what it charges for is what makes the results of Chapter 6 interpretable rather than merely quotable. Both predictive compressors in this thesis use exactly this coder.

3.2.1 What a residual is

Let $x[m]$ be the sample to be transmitted and let P be some rule — the *predictor* — that produces an estimate

$$\hat{x}[m] = P\big(x[m-L],\, x[m-2L],\, \ldots\big) \tag{3.1}$$

from samples that have *already been transmitted*. The encoder does not send $x[m]$. It sends the *residual*

$$d[m] = x[m] - \hat{x}[m]. \tag{3.2}$$

The decoder, having already reconstructed $x[m-1], x[m-2], \ldots$, runs the identical rule P on them, obtains the identical $\hat{x}[m]$, and recovers

$$x[m] = d[m] + \hat{x}[m]. \tag{3.3}$$

The lag L is the distance back along the ramp axis to the samples used for prediction; Section 3.3 explains why it is $L = n_{\mathrm{Tx}} = 12$ on this radar rather than 1.

Two things follow, and both matter.

Nothing is lost. Equation (3.2) is a *re-expression*, not an approximation. Given $\hat{x}[m]$, the map $x \mapsto d$ is a bijection. The sequence $\{d[m]\}$ carries exactly the information the sequence $\{x[m]\}$ carried — no more, no less. This is why a predictive coder can be lossless at all.

The predictor must be causal. P may only use samples the decoder already has, or side information sent ahead of them. A predictor that peeked at $x[m]$ itself would be unreproducible at the far end. It is also why Section 2.4.4 makes such a point of *which* version of the past the encoder uses — the originals or the reconstructions — since those differ the moment anything is not exactly lossless.

3.2.2 Why a re-expression can be worth anything

If no information is lost and none is gained, why is the residual sequence any cheaper to write down?

Because the *distribution* has changed, and a well-chosen code charges by distribution. Table 3.1 shows this on real data: one range bin of one receive channel, sixteen consecutive ramps, taken from frame 0 of the ColoRadar capture, predicted twelve ramps back by DRHE.

Table 3.1: The same sixteen real samples, written two ways. Range bin 75, receive channel 1, ramps 168–183. Both rows carry identical information.

m	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183
x (raw)	−69	−52	59	−6	−29	−15	59	−20	−69	−73	−45	84	−73	−50	59	−5
d (residual)	3	−3	0	−1	1	3	−4	3	−1	−2	−1	0	−2	4	−1	1

The raw values swing over $[-73, +84]$; writing any of them needs eight bits. The residuals never leave $[-4, +4]$; four bits would do. Since the two rows are informationally equivalent, that factor of two is free — provided the coder can be told about it.

And there is the catch. The bound $[-4, +4]$ holds for *these* sixteen samples. Somewhere else in the frame a target appears, or the scene changes, and a residual of 400 arrives. A fixed four-bit field cannot carry it. What is needed is a code that spends few bits on the common case, many on the rare one, and lets the decoder tell which is which.

3.2.3 The parsing problem

The naive approach fails immediately. Suppose the rule were "small values get four bits, large values get twelve". When the decoder reads four bits, it has no way to know whether it has just read a complete small value or the first third of a large one. Variable-length coding is not merely a matter of writing fewer bits; it is a matter of writing them so that the boundaries can be recovered.

There are two standard answers:

1. **A prefix-free code over every possible value.** Assign every one of the 65,536 possible 16-bit residuals its own codeword, no codeword being a prefix of another. This is optimal, and it is what a Huffman code over the raw alphabet would give — but it needs a 65,536-entry table, and the table has to be derived from the data and transmitted alongside it.
2. **Split the value into a size and a payload.** Send a small *description of how big the value is*, coded with a prefix-free code over a handful of symbols, followed by the value itself in exactly that many raw bits. The decoder reads the size first, so it always knows how many bits to read next.

Both compressors here take the second route, which is the same construction JPEG uses for DC coefficients. It is cheap in every respect that matters for hardware: the prefix-free table has sixteen entries rather than 65,536, it can be fixed at design time rather than derived per frame, and decoding a symbol is a sixteen-way comparison rather than a tree walk.

3.2.4 The category S_4

The "size" half of the split is the category

$$S_4(v) = \begin{cases} 0 & v = 0, \\ \min\bigl(15,\ \lfloor \log_2 |v| \rfloor + 1\bigr) & v \neq 0, \end{cases} \tag{3.4}$$

which is simply the number of bits needed to write $|v|$. Its ranges are given in Table 3.2.

The design is tight by construction: category k contains exactly 2^k values, so k payload bits identify one of them with nothing wasted and nothing left over. Section 5.4 deals with the one value for which "nothing left over" becomes a problem.

Table 3.2: The first few categories. Category $k \geq 1$ covers $|v| \in [2^{k-1},\, 2^k - 1]$, which counting both signs is exactly 2^k distinct values — precisely what k raw bits can enumerate.

S_4	values covered	count	
0	$v = 0$	1	the only category with no payload
1	$v \in \{-1, +1\}$	2	
2	$\lvert v\rvert \in \{2, 3\}$	4	
3	$\lvert v\rvert \in [4, 7]$	8	
4	$\lvert v\rvert \in [8, 15]$	16	
⋮			
15	$\lvert v\rvert \in [16384, 32767]$	32,768	exactly fills 15 bits

3.2.5 The APPEND bits

The payload half is

$$A(v) = \begin{cases} v & v \geq 0, \\ v + 2^{S_4(v)} - 1 & v < 0, \end{cases} \tag{3.5}$$

written in $S_4(v)$ raw bits.

The offset for negative values looks arbitrary but is doing something specific. Table 3.3 enumerates category 3 in full.

Table 3.3: Every codepoint of category 3. The negative values map to patterns with a clear most significant bit, the positive values to patterns with it set — so the sign is recovered from that one bit without transmitting a sign bit.

v	$A(v)$	3-bit pattern	MSB
-7	0	`000`	0
-6	1	`001`	0
-5	2	`010`	0
-4	3	`011`	0
$+4$	4	`100`	1
$+5$	5	`101`	1
$+6$	6	`110`	1
$+7$	7	`111`	1

Because positive values in category k span $[2^{k-1}, 2^k - 1]$ — exactly the k-bit patterns with the top bit set — and the offset maps the negatives onto $[0, 2^{k-1} - 1]$ — exactly those with it clear — the sign falls out of the payload for free. The decoder's rule is therefore

$$v = \begin{cases} A & \text{if the MSB of the } S_4 \text{ payload bits is set,} \\ A - \left(2^{S_4} - 1\right) & \text{otherwise,} \end{cases} \tag{3.6}$$

which in hardware is one bit test and one subtract.

3.2.6 Cost, and the shape of the dictionary

The total cost of coding one residual is

$$L(v) = \underbrace{\ell\big(S_4(v)\big)}_{\text{Huffman codeword}} + \underbrace{S_4(v)}_{\text{payload}}, \tag{3.7}$$

where $\ell(\cdot)$ is the codeword length for that category. The dictionary is *fixed* — not computed per frame — which saves transmitting it and removes a serial dependency from the hardware.

The *original* dictionary, taken from Kiem's thesis, has lengths

$$\ell = [\,4, 3, 2, 2, 2, 5, 6, 7, 8, 9, 10, 11, 14, 14, 13, 12\,] \text{ bits.} \tag{3.8}$$

The full dictionary, codewords included, is in Appendix A. Every one of the sixteen entries was verified against the published table [11, Table A.1]; all sixteen agree once the published most-significant-bit-first codes are reversed for the least-significant-bit-first packer used in this implementation.

Substituting into Equation (3.7) gives the cost curve of the original dictionary:

S_4	$\lvert v\rvert$ range	ℓ	payload	**total**
0	0	4	0	**4**
1	1	3	1	**4**
2	2–3	2	2	**4**
3	4–7	2	3	**5**
4	8–15	2	4	**6**
5	16–31	5	5	**10**
6	32–63	6	6	**12**
7	64–127	7	7	**14**
8	128–255	8	8	**16**

Three features of this curve deserve attention.

Everything below 16 is nearly free. A residual anywhere in $[-15, +15]$ costs between four and six bits. Against sixteen raw bits that is already a compression ratio between 2.7 and 4.0, achieved without the predictor doing anything clever — it only has to keep residuals small.

There is a cliff at category 5. Cost jumps from six bits to ten. A residual of 15 costs six bits; a residual of 16 costs ten. The coder is, by design, forgiving of small errors and severe about larger ones.

Above category 8 the code loses. A residual of 128 costs sixteen bits — exactly what the raw value would have cost. Beyond that the coder actively expands. This is not a flaw: it is the inevitable consequence of assigning short codewords to some symbols, and it is why the `random-int16` edge case of Section 4.5 expands the data by 67 %.

The dictionary's shape encodes an assumption about the data — that categories 2, 3 and 4 dominate, since those get the two-bit codewords — and it can never code a value in fewer than four bits. Section 3.6 shows that the prediction errors of this thesis are smaller than that, and derives dictionaries that match them.

3.2.7 Worked example 1: one value, in full

Take $v = -5$.

1. $\lvert v\rvert = 5$, and $\lfloor \log_2 5 \rfloor + 1 = 3$, so $S_4 = 3$.
2. $v < 0$, so $A(v) = -5 + 2^3 - 1 = 2$, written in three bits as `010`.
3. The original dictionary gives category 3 the codeword `00`, two bits.
4. The emitted bits are `00 010` — five bits, against sixteen for the raw value.

Decoding runs the same steps backwards. The decoder matches `00` against the dictionary, recovers $S_4 = 3$, reads the next three bits `010`, sees the most significant is `0` so the value was negative, and computes $2 - (2^3 - 1) = -5$.

3.3 Predicting on a time-division MIMO radar

Both predictors work along *slow time*: they predict a range bin's value in one ramp from its values in earlier ramps. Section 2.2.3 showed that on a TDM-MIMO cascade the ramp axis interleaves the transmitters, so ramp m and ramp $m-1$ come from different antennas eleven times in twelve. The two predictors are therefore applied at a lag of $L = n_{\mathrm{Tx}} = 12$ ramps: each ramp is predicted from the previous ramp of the *same* transmitter. This section shows on real data why that choice matters.

Table 3.4 traces sixteen consecutive ramps of one range bin — the same bin as Table 3.1 — through the DRHE predictor twice. The left half predicts from ramp $m-1$, which is how slow-time prediction is described for a single-transmitter radar. The right half predicts from ramp $m-n_{\mathrm{Tx}}$. Everything else — the filters, the coefficients, the entropy coder, the dictionary — is identical between the two halves.

Table 3.4: Sixteen real samples traced through DRHE at two prediction lags. Range bin 75, receive channel 1, frame 0, ramps 168–183; real part shown, the imaginary part behaves identically. The window is late enough in the frame that both predictors' filters are fully converged. The two halves differ only in which earlier ramp the prediction comes from.

			predicted from ramp $m-1$				**predicted from ramp $m-n_{\mathrm{Tx}}$**			
m	TX	x	$\hat{x}$	d	S_4	bits	$\hat{x}$	d	S_4	bits
168	0	−69	67	−136	8	16	−72	3	2	4
169	1	−52	−75	23	5	10	−49	−3	2	4
170	2	59	81	−22	5	10	59	0	0	4
171	3	−6	64	−70	7	14	−5	−1	1	4
172	4	−29	24	−53	6	12	−30	1	1	4
173	5	−15	−29	14	4	6	−18	3	2	4
174	6	59	−6	65	7	14	63	−4	3	5
175	7	−20	−83	63	6	12	−23	3	2	4
176	8	−69	−93	24	5	10	−68	−1	1	4
177	9	−73	−65	−8	4	6	−71	−2	2	4
178	10	−45	−44	−1	1	4	−44	−1	1	4
179	11	84	28	56	6	12	84	0	0	4
180	0	−73	70	−143	8	16	−71	−2	2	4
181	1	−50	−75	25	5	10	−54	4	3	5
182	2	59	81	−22	5	10	60	−1	1	4
183	3	−5	67	−72	7	14	−6	1	1	4
				total bits		**176**				**66**
				bits per value		11.00				4.12
				compression ratio		1.455				**3.879**

The left half is a predictor working across the wrong axis. Its residuals (-136, $+23$, -22, -70, …) are frequently *larger in magnitude than the samples they replace*, they land in categories 5 through 8, and they cost 11.00 bits per value — a compression ratio of 1.455, which is to say the entropy coder is salvaging a little from a predictor that has actively made things worse.

The right half is the same predictor twelve ramps back. Its residuals never exceed 4 in magnitude, they land in categories 0 through 3, and they cost 4.12 bits per value.

Why the difference is so large

Table 3.5 answers this directly, and it is worth studying because it is the mechanism behind the prediction gains reported in Chapter 6. DRHE (Section 3.4) predicts magnitude with a

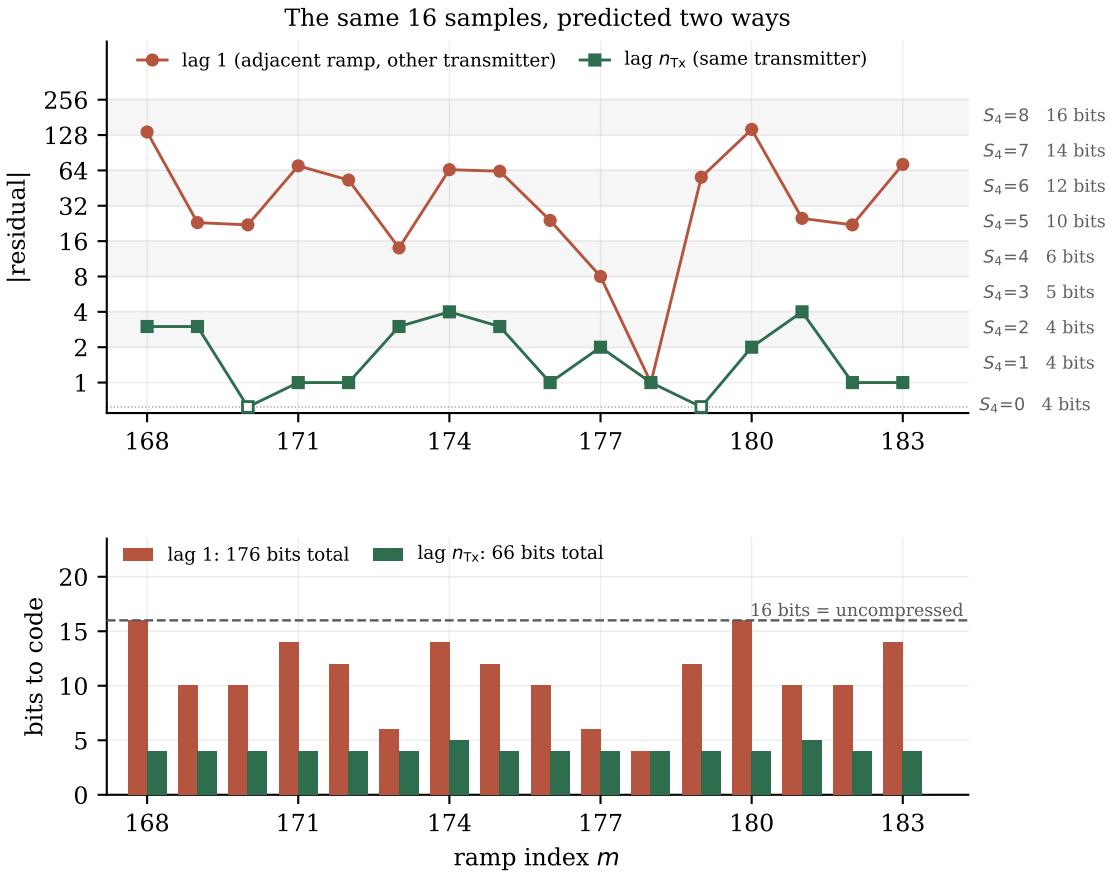


Figure 3.1: Table 3.4 drawn. Above: residual magnitude on a logarithmic scale, with the S_4 category bands and their bit costs marked on the right; hollow markers are residuals of exactly zero. Below: the bits each sample costs, against the 16 bits it would have taken uncompressed. The lag-1 predictor spends 176 bits on this window; the same predictor on the correct axis spends 66.

smoother and phase with a double integrator, both of which work when the quantity being predicted changes slowly and smoothly.

Look at the $\Delta\theta$ columns. Against ramp $m-1$ the phase increment is -2.244, then $+0.592$, then $+1.230$, then -0.678 — it swings over most of $[-\pi, \pi]$ with no pattern a two-tap filter could follow. Against ramp $m - n_{\mathrm{Tx}}$ it is 0.002, -0.040, 0.017, -0.008: essentially zero, frame after frame, because this is a stationary target and twelve ramps later the same antenna sees the same phase.

The magnitude column tells the same story: ± 22 against the immediate predecessor, ± 4 against the same-transmitter predecessor.

The reason is the one given in Section 2.2.3. Ramps 168 and 167 were transmitted by *different antennas*, so the phase between them is set by where the target is in *angle*, via Equation (2.6) — and the twelve transmitters are at twelve different positions, so that phase jumps around on a period-12 pattern. Ramps 168 and 156 were transmitted by the *same* antenna, so the phase between them is set by how fast the target is *moving*, via Equation (2.5) — and it is not moving.

The actual bits

It is worth seeing the bitstream itself. Coding the first six residuals of the lag-n_{Tx} column, using Equations (3.4) and (3.5) and the dictionary of Appendix A:

Table 3.5: The same window in polar coordinates. Against ramp $m-1$ the magnitude moves by up to 22 and the phase jumps unpredictably over the full $\pm\pi$ range. Against ramp $m-n_{\mathrm{Tx}}$ the magnitude moves by at most 4 and the phase increment is essentially zero — a stationary target, exactly what the double integrator is built for.

		sample		change vs. $m-1$		change vs. $m-n_{\mathrm{Tx}}$	
m	TX	$\lvert x\rvert$	$\angle x$	$\Delta\lvert x\rvert$	$\Delta\theta$	$\Delta\lvert x\rvert$	$\Delta\theta$
168	0	74.1	−2.769	−21.8	−2.244	−3.2	0.002
169	1	91.3	−2.177	17.2	0.592	−0.0	−0.040
170	2	101.0	−0.947	9.8	1.230	−2.5	0.017
171	3	110.2	−1.625	9.1	−0.678	3.0	−0.008
172	4	107.0	−1.845	−3.2	−0.220	−2.2	0.004
173	5	112.0	−1.705	5.0	0.140	−0.3	0.018
174	6	97.0	0.917	−15.0	2.622	−0.4	0.023
175	7	81.5	1.819	−15.5	0.902	0.4	−0.027
176	8	73.4	2.794	−8.1	0.975	−0.1	0.043
177	9	75.0	−2.913	1.6	0.576	2.4	0.021
178	10	95.3	−2.063	20.3	0.850	0.9	0.005
179	11	97.8	−0.537	2.5	1.526	1.9	−0.013
180	0	77.5	−2.799	−20.3	−2.263	3.4	−0.031
181	1	92.6	−2.141	15.2	0.659	1.4	0.036
182	2	102.6	−0.958	10.0	1.182	1.6	−0.011
183	3	106.1	−1.618	3.5	−0.659	−4.0	0.007

d	S_4	$A(d)$	codeword	payload	emitted	bits
+3	2	3	`10`	`11`	`10 11`	4
−3	2	0	`10`	`00`	`10 00`	4
0	0	—	`0101`	—	`0101`	4
−1	1	0	`001`	`0`	`001 0`	4
+1	1	1	`001`	`1`	`001 1`	4
+3	2	3	`10`	`11`	`10 11`	4

which concatenates to the 24-bit stream

`1011 1000 0101 0010 0011 1011`

carrying six samples that would have occupied 96 bits raw. (The codewords are shown in their stored, bit-reversed form; Appendix A lists both forms.)

The same six samples at lag 1 — residuals −136, +23, −22, −70, −53, +14 — fall into categories 8, 5, 5, 7, 6 and 4 and occupy $16+10+10+14+12+6=68$ bits. Nearly three times as many, for identical information.

3.3.1 Why not one model per transmitter?

An alternative way to keep prediction on one antenna is to split each frame into twelve independent 16-ramp blocks, one per transmitter, and predict within each. For DRHE that is equivalent to the lag-n_{Tx} indexing above. For LPC it is not: fitting separate coefficients to each block multiplies the side information by twelve, from 1.04 % to 12.5 % of the uncompressed frame, and shortens each autocorrelation estimate from 192 ramps to 16, which makes the coefficients much noisier. Measured over five frames it compresses to only 2.63, worse than not predicting at all. Keeping one coefficient pair per (bin, channel, part) and moving the lags to n_{Tx} and $2n_{\mathrm{Tx}}$ costs no extra side information, and is the design used here. On a TDM-MIMO radar the right change is to the *lag*, not to the *partitioning*.

3.4 DRHE: Doppler Redundancy Hybrid Encoding

DRHE [26, 25] exploits the redundancy of radar data along the Doppler (slow-time) axis, and is a *hybrid* of prediction and entropy coding: a predictor removes the slow-time structure, and an entropy coder — originally run-length plus Huffman, here the S_4 coder of Section 3.2 — codes what is left.

3.4.1 Why predict in polar coordinates

Section 2.1 established what a range-bin sequence looks like across slow time for a target at constant velocity: the *magnitude* is nearly constant, and the *phase* advances by a constant increment. In Cartesian (I, Q) coordinates that is a point rotating on a circle — both components oscillate, and a simple difference between consecutive samples is large. In polar coordinates the same sequence is two nearly-trivial signals: a constant and a ramp.

DRHE therefore converts to polar, predicts magnitude and phase separately with filters matched to those two shapes, converts the prediction back to Cartesian, and subtracts.

3.4.2 The two predictors

For ramp m and range bin n, with $a[m,n] = |x[m,n]|$ the magnitude and $\theta[m,n] = \angle x[m,n]$ the phase, and $L = n_{\mathrm{Tx}}$ the prediction lag of Section 3.3, the predictions are

$$\hat{a}[m,n] = \alpha\,\hat{a}[m-L,n] + (1-\alpha)\,a[m-L,n], \tag{3.9}$$

$$\hat{\theta}[m,n] = \beta\,\hat{\theta}[m-L,n] + (2-\beta)\,\theta[m-L,n] - \theta[m-2L,n], \tag{3.10}$$

with $\alpha = 0.6$ and $\beta = 0.4$ [11, Table 4.4], and $\hat{\theta}$ wrapped into $[-\pi, \pi]$.

Equation (3.9) is a first-order infinite-impulse-response smoother: it blends the previous prediction with the previous observation. For a constant magnitude it converges to that constant, and the α term gives it some immunity to noise on any single ramp.

Equation (3.10) is more interesting. Setting $\beta = 0$ reduces it to $\hat{\theta}[m] = 2\theta[m-L] - \theta[m-2L]$, which is a *double integrator*: it extrapolates the last two phases along a straight line. For a phase advancing by a constant $\Delta\theta$ per ramp — exactly Equation (2.5) — that prediction is perfect. The β term mixes in the previous prediction to damp the response to noise.

The prediction is then converted back and subtracted, with int16 wraparound $\mathcal{W}(\cdot)$:

$$d_{\Re} = \mathcal{W}\big(x_{\Re} - \mathrm{round}(\hat{a}\cos\hat{\theta})\big), \tag{3.11}$$

$$d_{\Im} = \mathcal{W}\big(x_{\Im} - \mathrm{round}(\hat{a}\sin\hat{\theta})\big), \tag{3.12}$$

and $d_{\Re}$, $d_{\Im}$ are handed to the entropy coder of Section 3.2.

3.4.3 The streaming property

DRHE has a property that matters more than its compression ratio: it is *causal and streaming*. Each output depends only on the current sample and on state carried from earlier ramps of the same transmitter. Concretely, the implementation holds five floating-point values per (transmitter, receive channel, range bin) — the magnitude prediction, the previous magnitude, the phase prediction, and the previous two phases — and nothing else. That is 120 KiB of state for this radar, against 384 KiB for the frame itself.

It therefore needs no frame buffer, and it emits its first compressed bits as soon as the first sample arrives. For a compressor sitting between an FFT and a memory bus, that is exactly the right shape. Section 7.1 returns to this when weighing DRHE against LPC.

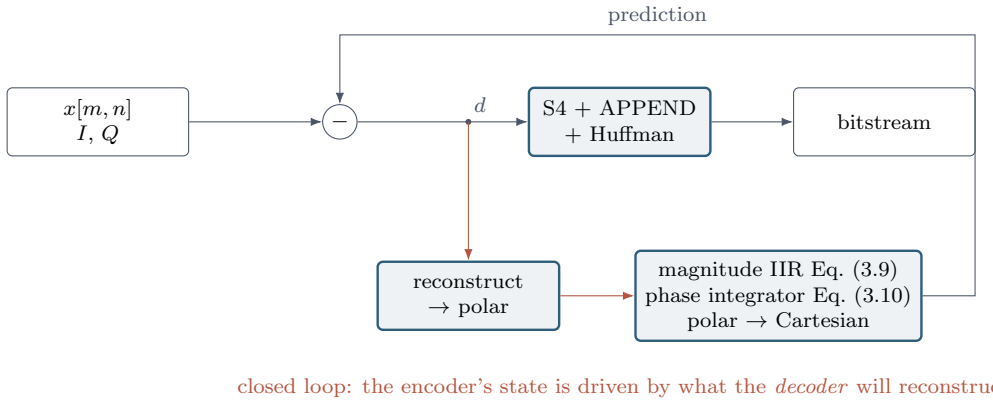


Figure 3.2: DRHE as implemented. Prediction happens in the polar domain with two filters matched to the expected behaviour of a constant-velocity target; the prediction is converted back to Cartesian before subtraction. The red path is the closed-loop correction of Section 5.4: the encoder drives its state from what the decoder will reconstruct, not from the original sample.

3.5 LPC: linear prediction with Yule–Walker coefficients

3.5.1 Autoregressive modelling

The second algorithm takes a more general approach. Rather than assuming a particular signal shape and hand-designing filters for it, it *fits* a model to the data.

A second-order autoregressive (AR) model says each sample is approximately a linear combination of the previous two:

$$\hat{x}[m] = a_1 x[m - L] + a_2 x[m - 2L], \tag{3.13}$$

Given a sequence, the coefficients that minimise the mean-square prediction error satisfy the Yule–Walker equations, which for order two are

$$\begin{bmatrix} R_0 & R_1 \\ R_1 & R_0 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} R_1 \\ R_2 \end{bmatrix}, \tag{3.14}$$

where $R_k = \sum_m x[m]\, x[m - kL]$ is the autocorrelation at lag kL, summed over every ramp of the frame. Solving directly,

$$a_1 = \frac{R_1(R_0 - R_2)}{R_0^2 - R_1^2}, \qquad a_2 = \frac{R_0 R_2 - R_1^2}{R_0^2 - R_1^2}. \tag{3.15}$$

3.5.2 Stability

An AR model whose poles lie outside the unit circle is unstable: prediction errors grow rather than decay. For order two the stability triangle gives the conditions

$$|a_2| < 1 \qquad \text{and} \qquad |a_1| < 1 - a_2. \tag{3.16}$$

If either fails, or if the denominator $R_0^2 - R_1^2$ vanishes, the implementation sets $a_1 = a_2 = 0$, which degrades gracefully to "predict zero" — i.e. code the raw value. On the real data used here the test never fires: 100.00 % of coefficient sets pass.

3.5.3 Granularity and side information

The model is fitted per *(range bin, receive channel, real/imaginary part)*, across all ramps of the frame. That granularity is a deliberate compromise: fitting one model to the whole frame would ignore that different range bins contain targets at different velocities, while fitting per (bin, ramp) would have no data to fit to.

The coefficients are side information: the decoder cannot reproduce them, so they must be transmitted. Two 16-bit values per (bin, channel, part) is

$$128 \times 4 \times 2 \times 2 \times 16 = 32{,}768 \text{ bits} \tag{3.17}$$

per frame, which is 1.04 % of the uncompressed frame. That is a small but real tax, and Section 3.3 shows why fitting more coefficients — one set per transmitter, for instance — makes it the dominant cost.

3.5.4 The cost: LPC is not causal

R_0, R_1 and R_2 are sums over *all* ramps of the frame. The coefficients therefore cannot be known until the frame is complete, and the compressor cannot emit a single bit before then. Where DRHE holds five values per (channel, bin), LPC must buffer the entire frame: $4 \times 192 \times 128 \times 32 = 3{,}145{,}728$ bits, 3.0 Mibit.

This is the fundamental trade between the two algorithms, and Section 7.1 argues it matters more than the difference in compression ratio.

3.5.5 Coefficient quantisation

The Yule–Walker solution of Equation (3.15) is computed in floating point and then quantised before use, because the decoder must use exactly the coefficients the encoder used and must be able to read them from the bitstream. a_1 is stored as `ap_fixed<16,2>` (range $[-2, 2)$, step 2^{-14}) and a_2 as `ap_fixed<16,1>` (range $[-1, 1)$, step 2^{-15}), both with truncation towards $-\infty$, which is the HLS default. The prediction is then formed in fixed point and rounded to an integer before the residual is taken, so encoder and decoder agree to the bit. The MATLAB model reproduces this quantisation; the independent Python implementation of Appendix D does too, and matches the HLS bit counts exactly on all 50 frames.

3.5.6 What this LPC is not

Kiem's thesis mentions an "LPC-Huffman" method [27] that he judged less promising than DRHE and did not implement. That method predicts each raw ADC sample from earlier samples of the *same chirp* — along fast time, before the range FFT — and codes the residuals with a Huffman code derived from the data. The LPC here predicts each range-FFT code from the same range bin in earlier *ramps* — along slow time, after the range FFT, exactly where DRHE predicts — and uses DRHE's own entropy coder. It was designed that way on purpose: by keeping DRHE's input, axis and coder and changing only the predictor, any difference in compression ratio can be attributed to the predictor alone. Section 7.6 returns to how the two methods called LPC compare.

3.6 The Huffman dictionary

The entropy coder needs one design-time constant: the sixteen codeword lengths $\ell_0, \ldots, \ell_{15}$, one per S_4 category. Two dictionaries are used in this thesis.

Table 3.6: The two lossless algorithms compared on structure, before any measurement.

	DRHE	**LPC**
Prediction domain	polar (magnitude, phase)	Cartesian, I and Q independently
Model	fixed IIR filters, α, β chosen a priori	second-order AR, coefficients fitted per frame
Prediction lag	n_{Tx}	n_{Tx} and $2n_{\mathrm{Tx}}$
Causal?	yes — streaming, first output immediate	**no** — needs the whole frame before any output
Compressor state	5 values per (TX, channel, bin)	full frame, 3.0 Mibit
Side information	none	32,768 bits/frame (1.04 %)
Per-sample arithmetic	4 floating-point transcendentals (`sqrt`, `atan2`, `sin`, `cos`)	2 fixed-point multiply–accumulates
Per-frame arithmetic	—	one division per (channel, bin, part)
Entropy back end	identical: S_4 + APPEND + Huffman dictionary	

The original dictionary. Kiem derived his dictionary once, from the S_4 statistics of one of his own recordings, and fixed it for simulation and hardware. Its two-bit codewords go to categories 2, 3 and 4, so it is matched to residuals of magnitude 2 to 15 — the size his single-transmitter data produced. No value can be coded in fewer than four bits, which caps the compression ratio of any frame at $32/(2 \times 4) = 4.0$.

The re-derived dictionaries. The prediction errors of DRHE and LPC at lag n_{Tx} are smaller than that: most fall in categories 0, 1 and 2 (Section 6.4). A new set of codeword lengths was therefore derived for each algorithm, by building a Huffman code [15] on the S_4 category counts of the algorithm's own residuals over frames 0–4 only. One is added to every count before the code is built, so that categories never seen in training still receive a codeword and the code remains lossless on any input. The lengths are assigned as canonical codes and, like the original, bit-reversed for the least-significant-bit-first packer:

$$\begin{aligned}
\ell^{\mathrm{orig}} &= [\,4,3,2,2,2,5,6,7,8,9,10,11,14,14,13,12\,],\\
\ell^{\mathrm{DRHE}} &= [\,3,2,1,4,5,6,7,8,9,10,11,14,14,13,13,13\,],\\
\ell^{\mathrm{LPC}} &= [\,3,1,2,4,5,6,7,8,9,10,13,13,13,13,12,12\,].
\end{aligned}$$

DRHE's one-bit codeword goes to category 2 and LPC's to category 1, so their commonest residuals cost three bits and two bits respectively, rather than four. Because only the contents of a sixteen-entry table change, a re-derived dictionary costs nothing in hardware; the FPGA cores of Chapter 5 use them. Both dictionaries are listed in full in Appendix A, and Section 6.4 reports the difference they make, including on the 45 frames they were not trained on.

3.7 The two algorithms side by side

Table 3.6 summarises the two designs. The trade they present is clear before anything is measured: LPC fits its model to the data and so should predict at least as well as DRHE's fixed filters, but it pays for that with side information and, more importantly, by having to see the whole frame before it can emit anything. DRHE assumes a signal shape and pays for it in floating-point arithmetic, but it streams. Chapter 6 measures how large each of these effects is.

Chapter 4

Evaluation Method

This chapter sets out how the methods of Chapter 3 were evaluated: the two stages of the evaluation, the signal chain that produces their input, the metrics, and the verification ladder that carries every result from MATLAB down to the FPGA board.

4.1 Two stages

The evaluation runs in two stages, each answering one question.

Stage 1 — which methods are worth building? All eight methods of Section 3.1 are run in MATLAB over the same 50 consecutive ColoRadar frames with all 16 receive channels. Each is scored on compression ratio and on what it does to target detection. A method goes forward only if it is lossless — it loses no detections beyond those of the FX16 step every method shares — and compresses by more than three, which leaves a margin over the 2.4–2.7 that a Gigabit Ethernet link needs (Section 2.9).

Stage 2 — where does the compression come from, and what does it cost in hardware? The selected methods, DRHE and LPC, are evaluated on the four-receiver frames that the hardware processes. Their compression ratio is broken down into what the entropy coder achieves alone, what prediction adds, and what a re-derived dictionary adds. They are then implemented, with decoders, in hardware (Chapter 5), verified bit for bit at every level of the ladder of Section 4.4, and measured on the board for time per frame, latency and chip resources.

4.2 The reference signal chain

The MATLAB chain is described here in the order it executes, because several later observations depend on details of individual steps.

4.2.1 Loading

A frame is read from the ColoRadar binary and reshaped into a complex cube of (fast time $\times$ ramp $\times$ receiver), with the TDM flattening of Equation (2.7). The per-ramp mean is subtracted from each channel to remove the DC / transmitter-leakage term.

4.2.2 Windowing

A Hann window is applied along fast time to suppress spectral leakage, so that a strong nearby target does not bury a weak distant one in sidelobes.

4.2.3 Range FFT

An N_s-point FFT is taken along fast time and normalised by N_s. Only the positive half-spectrum is retained, giving $N = 128$ range bins. This halving is a deliberate choice, not a consequence of conjugate symmetry — the input here is complex I/Q, so the spectrum is *not* symmetric; the negative half corresponds to negative beat frequencies, which are not physically meaningful ranges.

4.2.4 FX16 quantisation

The complex FFT output is scaled by Equation (2.11) and rounded to `int16`. This is the last step before compression, and its output — 128 range bins × 192 ramps × 4 receive channels of complex `int16` — is exactly what both compressors consume.

Four receive channels rather than sixteen are exported to the hardware path, because four complex `int16` samples is 128 bits, the AXI4-Stream width. One such frame is $128 \times 192 \times 4$ complex samples, or 393,216 B; the 50-frame test set is 19.7 MB. Figure 4.1 shows one receiver of one such frame.

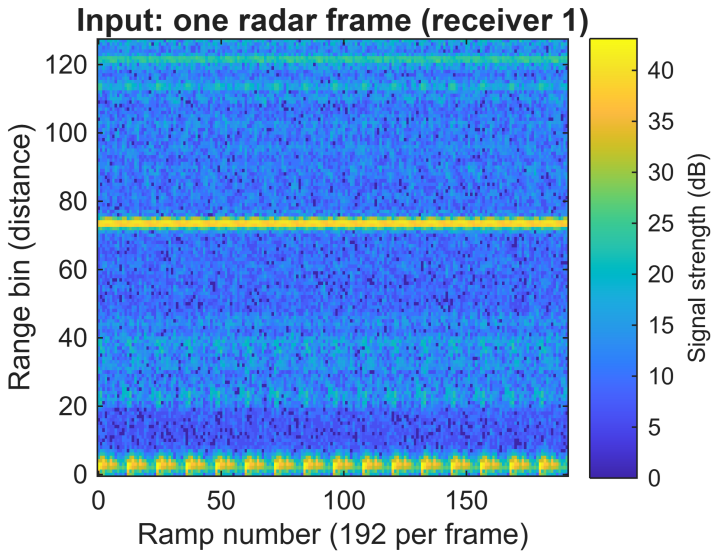


Figure 4.1: One input frame as the compressor receives it: receiver 1 of ColoRadar frame 0 after the range FFT and FX16 quantisation, 128 range bins by 192 ramps. Strong returns appear as horizontal bands at fixed range; most of the frame is low-level noise, which is why it compresses.

4.2.5 Detection and metrics

The reconstructed data is put through a Doppler FFT and a threshold detector, and the resulting detections are compared against those from the double-precision reference. Three details of this chain matter for how its results should be read:

- the Doppler FFT is a 192-point FFT across *all* ramps of the frame, in the order they were recorded — that is, across the transmitter-interleaved ramp axis of Section 2.2.3, not across the 16 ramps of any one transmitter;
- there is no angle FFT: the 16 receive channels are combined by non-coherent integration (summing magnitudes), so "detection" here means a peak in a range–Doppler map, not a localised point in space;
- the threshold is the mean of the lowest 75 % of sorted magnitudes plus a fixed 15 dB offset, with 8-connected local-maximum suppression.

Neither of the first two affects any lossless result, since a lossless compressor returns the input exactly and every downstream stage then agrees by construction. They matter only for the lossy comparison points, whose detection losses are measured by a chain that is simpler than a production one (Section 7.7).

4.3 Metrics, defined precisely

Several metric names are used loosely in this field. They are pinned down here.

4.3.1 Compression ratio

$$\mathrm{CR} = \frac{B_{\mathrm{in}}}{B_{\mathrm{out}}}, \qquad B_{\mathrm{in}} = N_{\mathrm{ramps}} \times N \times N_{\mathrm{RX}} \times 32, \tag{4.1}$$

with B_{out} the compressed length *including* AXI packet padding. Including the padding matters: a design that flushed its bit buffer per ramp rather than per frame would waste up to 255 bits per ramp, costing about 2.9 % of the ratio. Reporting the ratio without padding would hide that design choice. Ratios are computed per frame and reported as the mean over the 50 frames; the ratio of the totals differs by about 0.2 % (for example 4.208 against a mean of 4.217 for DRHE).

4.3.2 Maximum absolute difference

$$\mathrm{MaxDiff} = \max_{\text{all samples}} \left| x - \tilde{x} \right| \tag{4.2}$$

over every real and imaginary component. For a lossless coder the pass criterion is MaxDiff $= 0$. There is no tolerance.

4.3.3 Detection metrics

Let D_{ref} be the detection set from the double-precision reference and D_{test} that from the reconstructed data. The two rates reported are

$$\mathrm{FN\%} = \frac{|D_{\mathrm{ref}} \setminus D_{\mathrm{test}}|}{|D_{\mathrm{ref}}|}, \qquad \mathrm{FP\%} = \frac{|D_{\mathrm{test}} \setminus D_{\mathrm{ref}}|}{|D_{\mathrm{test}}|}. \tag{4.3}$$

The first is a false negative rate in the ordinary sense: the share of real targets that are missed, reported as *missed targets*. The second is labelled a false positive rate in Kiem's thesis, but it normalises by the number of *test detections* rather than by the number of true negatives, which makes it a **false discovery rate** (FDR): the share of reported detections that are spurious. It is reported under that name here.

Two further properties bound how much weight the detection metrics can carry, and are stated rather than buried:

- **The comparison is cell-wise and exact.** A target detected one range–Doppler cell away from its reference position counts as both a false negative *and* a false discovery. There is no positional tolerance, which inflates both rates relative to a matched-detection criterion.
- **The 15 dB threshold is a free parameter.** It is not swept, and the detection rates are sensitive to it.

4.3.4 What the detection metrics actually measure here

This point is important enough to state on its own. FX16, DRHE and LPC are all exactly lossless *with respect to the FX16 codes*. Their reconstructed data is bit-identical, so their detection metrics are identical *by construction* — not by coincidence, and not as evidence that the compressors are equally good.

What the detection metrics measure for the lossless methods is therefore the cost of the **FX16 quantisation step**, which is the only lossy operation in the lossless chain. For the lossy methods they measure the cost of quantisation and compression together.

4.3.5 No-prediction baseline and order-0 entropy

To separate what the predictor contributes from what the entropy coder contributes, every predictive result is reported against a *no-prediction baseline*: the FX16 codes fed straight into the same entropy coder. A predictor that does not beat it is not helping, however good the overall ratio looks. In the same spirit, Equation (2.9) is evaluated empirically on the residual distribution: the entropy is the figure of merit for the *predictor*, and the gap between it and the achieved bits per value is the inefficiency of the *dictionary*.

4.3.6 Timing

Three timing quantities are reported for the hardware, and they answer different questions.

- **Time per frame**: clock cycles from the start of a frame's DMA transfer to the last output word being written, divided by the clock. It sets the frame rate the design can sustain.
- **Throughput**: input bits per frame divided by the time per frame.
- **End-to-end latency**: the time from the last input word being read from memory to the last output word being written back, measured by the board firmware with a hardware timer. This is Kiem's definition, and it measures how long the compressed data lags the raw data — the quantity that matters for a sensor feeding a real-time system.

For a streaming design latency is a small fraction of the time per frame; for a design that must see the whole frame first, it is most of it.

4.4 The verification ladder

A compressor that is *nearly* lossless is useless, so the method has to prove exactness rather than estimate it. Every design is therefore taken down a ladder of five levels, each checked against the one above it, so that a discrepancy always localises to a single transition (Figure 4.2).

Level 1 — MATLAB. The whole radar chain is modelled in double precision and every method is run against it. This establishes the golden reference: the exact FX16 values, the exact number of bits each algorithm should produce, and the detection metrics.

Level 2 — HLS C-simulation. Because the hardware is written in C++, it can be compiled and run as ordinary software. This level asks whether the C++ produces *exactly* the bits MATLAB says it should, and whether the decompressor recovers *exactly* the original samples, over all 50 frames. It runs in minutes. The stimulus is exported from MATLAB as `int16` rather than recomputed in C++, so both environments operate on byte-identical input and any difference in output is unambiguously a difference in the compressor.

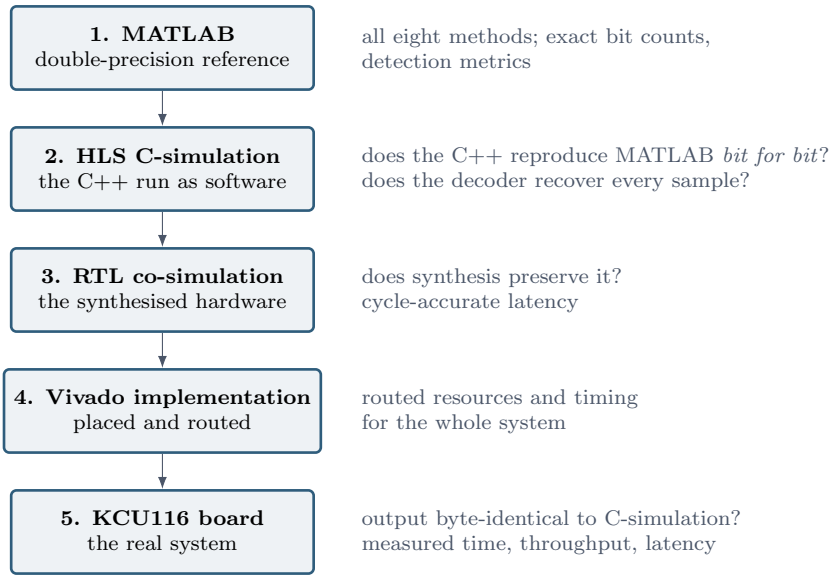


Figure 4.2: The verification ladder. Each level is checked against the one above it, so a mismatch localises to a single transition rather than to "somewhere in the design".

Level 3 — RTL co-simulation. The C++ is synthesised to Verilog and the same testbench is re-run against the generated RTL. This checks that synthesis preserved behaviour — pipelining, array partitioning and fixed-point inference can all change results if a design is subtly wrong — and gives a cycle-accurate latency figure. Co-simulation is run on one frame, since the per-frame behaviour is deterministic and 50 frames of RTL simulation with floating-point IP take many hours.

Level 4 — Vivado implementation. Each core is packaged as IP and built into the complete board system (Section 5.6), then synthesised, placed and routed by Vivado. This replaces the HLS resource and clock *estimates* with the figures of a real netlist: post-route utilisation per core and for the whole system, and the worst timing slack at 100 MHz.

Level 5 — the board. The bitstream runs on an AMD KCU116 evaluation board. All 50 frames are streamed through the compressor by DMA, the compressed output is compared byte for byte with the C-simulation output, the decompressed output is compared with the original frames, and the firmware times every frame with a hardware timer.

4.5 Edge-case testing

Real data exercises the typical case well and the extreme case not at all. The ColoRadar residuals span roughly $[-514, +416]$, which is entirely inside category 10 of a 16-category code. Everything above that is never exercised by real data.

An eight-case synthetic testbench was therefore built (Table 4.1), generating its stimulus internally so that it runs identically in C-simulation and co-simulation.

The suite exercises the parts of the design that real data never reaches, including the two situations handled by the robustness measures of Section 5.4, neither of which can arise on ColoRadar data.

Table 4.1: The synthetic edge-case suite. Each case has its own allowed reconstruction error — zero for all but the three that can produce the value −32768 and therefore exercise the saturation described in Section 5.4. Uniformly random `int16` input produces a residual of −32768 by chance, so it is one of them.

Case	What it exercises	Allowed MaxDiff
all-zeros	the $S_4 = 0$ path and the best case	0
dc-constant	a perfectly predictable sequence	0
random-int16	the worst case: incompressible input	1
tone-192ramp	a periodic sequence the AR model should fit	0
tone-1ramp	a sequence too short to fit	0
max-amplitude	full-scale values, and 128-bit correlations	0
max-alternating	±32767 alternating: the largest residuals	1
int16-min	−32768 specifically	1

4.6 An independent implementation

As a final check that the compression figures are right — not merely consistent between MATLAB and C++, which share a specification and could share a misunderstanding — the whole compression chain was written a third time, independently, in Python and NumPy, from the descriptions in Chapters 3 and 4 without reusing any code. It reads raw ColoRadar frames, applies the TDM flattening, DC removal, window, range FFT and FX16 scaling, runs DRHE and LPC with the same coefficient quantisation as the hardware, and counts the bits of the S_4, APPEND and Huffman coder with and without 256-bit padding. With the original dictionary it reproduces the HLS bit counts of both algorithms exactly on every one of the 50 frames, and with the re-derived dictionaries it reproduces the hardware's residuals exactly on the frames checked. It is also the source of the no-prediction baseline, the entropy figures and the LPC coefficient statistics reported in Chapter 6. Appendix D lists what was checked.

4.7 Reproducing the baseline

Kiem's recordings and source code are not public, so his numbers cannot be re-run directly. Two reconstructions connect them to this thesis, and Section 7.6 and Section 7.5 report what they show.

His simulation. His MATLAB chain was rebuilt from the text of his thesis — pre-processing, range FFT, FX16, DRHE with and without run-length coding, Doppler FFT and detection — and run on the synthetic five-target scene his thesis describes. The noise level is the one parameter his text does not pin down, so it was swept, and the chain was also used to run the fast-time LPC-Huffman method he excluded.

His hardware. His compressor was rebuilt from its description — the same fixed-point types, functions, state arrays and packing — in the same tool flow and on the same device as this thesis's DRHE, and connected to it by a ladder of variants that each change one design decision (arithmetic, state width, state reset, packer). Every variant was implemented through placement and routing and checked lossless, so each step's change in resources has one identified cause.

4.8 Tools

All MATLAB work used R2026a with the Signal Processing Toolbox. All hardware work used AMD Vitis HLS 2026.1 and Vivado 2026.1, targeting the `xcku5p-ffvb676-2-e` device of the KCU116 at a 100 MHz clock, with XSIM for co-simulation and Vitis for the MicroBlaze firmware. The board is controlled from a host PC over JTAG with `xsdb` scripts, and results are checked with Python. Appendix C lists the source files and build commands.

Chapter 5

Hardware Implementation

Both algorithms were implemented as a compressor core and a matching decompressor core in C++ for Vitis HLS, and built into a test system on an AMD KCU116 evaluation board. This chapter describes the cores, the measures that keep them exactly lossless on any input, and the board system that runs and measures them.

5.1 Common interface and bit packing

All four cores share one interface. A compressor accepts AXI4-Stream input at 128 bits per beat — four receive channels, each a 16-bit real and a 16-bit imaginary part — and emits a compressed AXI4-Stream at 256 bits per beat, with TLAST marking the end of the frame. A decompressor does the reverse. Frame dimensions (range bins, ramps, receive channels and n_{Tx}) arrive on an AXI4-Lite control interface, so there is no header in the bitstream.

Both compressors share the bit packer. Codewords accumulate in a 512-bit register, least-significant-bit first, and a 256-bit word is written out whenever at least 256 bits are waiting, with one final partial word at the end of the frame. Flushing per frame rather than per ramp avoids wasting up to 255 bits per ramp, about 2.9 % of the ratio. The Huffman codewords are stored bit-reversed relative to their usual most-significant-bit-first form, because the packer is least-significant-bit first; the reversed code set is still prefix-free, so a decoder can match candidates from the low end of its buffer with no ambiguity. Compressor and decompressor read the same sixteen-entry table. Each core was built with both dictionaries of Section 3.6; the two builds differ only in the contents of this table, and have the same schedule and timing. The board measurements of Chapter 6 were made with the original-dictionary builds.

5.2 DRHE compressor

5.2.1 State and parallelism

DRHE holds five `float` state arrays — the magnitude prediction, the previous magnitude, the phase prediction and the previous two phases — each indexed by (transmitter, receive channel, range bin). The transmitter index is the ramp number modulo n_{Tx}, which is all it takes to put the predictor on the per-transmitter axis of Section 3.3: indexing by transmitter changes *which* address is read, not *when*, so the schedule is the same as for a lag-1 design. Each transmitter's history starts on its own first ramp, so the state is reset while $m < n_{\mathrm{Tx}}$. Frames are independent: nothing carries over from one frame to the next, so a frame lost in transit cannot corrupt its successors.

The channel dimension is completely partitioned, so the four receive channels are four parallel copies of the datapath inside one pipelined iteration of the sample loop. The state

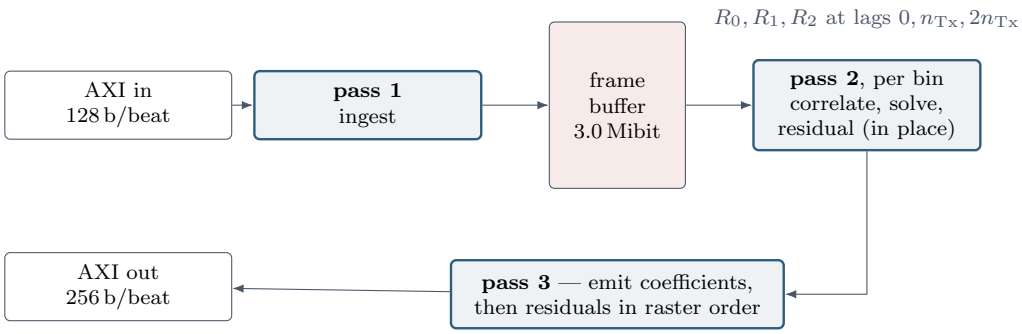


Figure 5.1: The LPC compressor. Because the Yule–Walker coefficients depend on the whole frame, the compressor must buffer it; each (channel, bin) is then correlated, solved and turned into residuals, which overwrite the buffer in place so that only one memory is needed. Emitting residuals in the original raster order is what lets the *decompressor* avoid a frame buffer entirely.

occupies 40 block RAMs (36 Kb).

5.2.2 Where the logic goes

Per sample and per channel, DRHE evaluates `sqrt` and `atan2` to convert the reconstructed sample to polar form, and `sin` and `cos` to convert the prediction back — four floating-point transcendental functions, for each of four channels, on every one of the 24,576 beats of a frame. These operators dominate the design: they account for about two thirds of its LUTs and all 132 of its DSP slices, and the two arctangent units alone are about a third of the logic. Kiem's design uses fixed point for the same functions; Section 7.5 compares the two.

5.2.3 Initiation interval

DRHE runs at $\mathrm{II} = 2$, and the reason has nothing to do with arithmetic. Each state array is partitioned by channel, so each channel's history is one dual-port block RAM. Every iteration reads a value and writes the updated one — two accesses, which two ports can serve. On the first ramps of a frame the iteration also writes a reset value: three accesses against two ports, which forces the tool to serialise, and the loop settles at $\mathrm{II} = 2$ for the whole frame. Reading the state as zero on the first ramps instead of writing zeros to it reaches $\mathrm{II} = 1$. It was not adopted, because it doubles the floating-point arithmetic (units can no longer be shared between two channels) and the sensor does not need the throughput: the $\mathrm{II} = 2$ design already runs many times faster than the radar produces frames (Section 6.7).

5.3 LPC compressor

5.3.1 Three passes

LPC cannot stream, so its structure is explicitly multi-pass (Figure 5.1):

1. **Ingest.** Read the whole frame into a buffer.
2. **Correlate, solve and residual**, once per (channel, bin): compute R_0, R_1 and R_2 at lags 0, n_{Tx} and $2n_{\mathrm{Tx}}$ from the buffer, solve Equation (3.15), then walk the ramps again computing residuals and writing them back into the buffer *in place*.
3. **Emit.** Write the coefficient table, then the coded residuals.

Rewriting the buffer in place means one 3.0 Mibit memory serves both purposes rather than two.

5.3.2 Reaching back n_{Tx} and $2n_{\mathrm{Tx}}$ ramps

Both the correlation and the residual loops need $x[m - n_{\mathrm{Tx}}]$ and $x[m - 2n_{\mathrm{Tx}}]$. A shift register of depth $2n_{\mathrm{Tx}} = 24$ would move 24 entries every iteration. Instead a *ring* of exactly $2n_{\mathrm{Tx}}$ entries gives both taps for free. At ramp m, let $k = m \bmod 2n_{\mathrm{Tx}}$. Slot k still holds the value written at ramp $m - 2n_{\mathrm{Tx}}$, because it is about to be overwritten and has not been touched since; and slot $(k + n_{\mathrm{Tx}}) \bmod 2n_{\mathrm{Tx}}$ holds the value from ramp $m - n_{\mathrm{Tx}}$. So:

read $x[m - 2n_{\mathrm{Tx}}]$	from slot k
read $x[m - n_{\mathrm{Tx}}]$	from slot $(k + n_{\mathrm{Tx}}) \bmod 2n_{\mathrm{Tx}}$
write $x[m]$	to slot k

Nothing is shifted; the cost is $2n_{\mathrm{Tx}}$ registers and two multiplexers. Every loop in the design reaches $\mathrm{II} = 1$.

5.3.3 Where the cycles go

A frame takes 325,366 cycles on the board. Ingest and emit are 24,576 cycles each, one per beat. The remaining 276,000 or so are the per-bin pass, which runs two short pipelined loops — correlation and residual — for each of the 512 (channel, bin) pairs. A pipelined loop pays its pipeline depth once on entry before reaching steady state, about 71 cycles for the correlation loop and 56 for the residual loop, so 512 invocations add roughly $512 \times (71 + 56) \approx 65{,}000$ cycles of fill that a trip-count model does not show. Merging the two loops, or processing several bins per invocation, would recover most of it; overlapping the three passes across frames would bring the time per frame down further. Neither was necessary for this thesis, and both are listed as future work.

5.3.4 Fixed-point choices

Coefficients are quantised to `ap_fixed<16,2>` for a_1 (range $[-2, 2)$) and `ap_fixed<16,1>` for a_2 (range $[-1, 1)$), which follows from the stability conditions of Equation (3.16). Over 50 frames this quantisation costs nothing measurable: the residuals are identical to the bit to those of double-precision coefficients.

The autocorrelations are accumulated as exact 64-bit integers rather than in floating point. $R_0^2 - R_1^2$ is a difference of two nearly equal quantities whenever the data is strongly correlated — precisely the regime the predictor exists for — so computing it in floating point would lose most of its significant digits exactly when accuracy matters most. The numerators and the determinant are formed exactly as 128-bit integers before the single division.

5.3.5 An asymmetry that favours the decoder

Pass 3 emits the coefficient table first — the decoder needs all of it before it can reconstruct anything — and then the residuals *in the raster order the samples arrived in*, not in the (channel, bin, ramp) order pass 2 produced them. The decompressor therefore needs no frame buffer at all: it holds the coefficient table and the last $2n_{\mathrm{Tx}}$ reconstructed samples of each (channel, bin), and writes each output sample as its residual arrives. The reordering is paid for once, by the side that already had to buffer. LPC's non-causality is a compressor-side cost only.

5.4 Exactly lossless on any input

Real radar data exercises only the lower part of the entropy coder's range (Section 4.5). Two situations that real data never produces, but synthetic input does (Section 4.5), are handled explicitly so that both designs are lossless on *any* input.

5.4.1 The one value the code cannot represent

Category 15 of the S_4 code covers $|v| \in [16384, 32767]$: counting both signs, exactly the 2^{15} patterns its fifteen payload bits can hold. But 16-bit arithmetic can produce $v = -32768$, for which Equation (3.5) gives $A(-32768) = -1$, which truncated to fifteen bits is `0x7FFF` — the same payload as $v = +32767$. The two would be indistinguishable on the wire.

Three remedies were considered: extend the code with a category 16, which changes the bitstream format; reject the input; or saturate -32768 to -32767. Saturation was chosen. It costs at most one least-significant bit on the affected sample, leaves the format untouched, and never fires on real data.

Saturation alone is not enough, though, and this is where the closed-loop prediction of Section 2.4.4 becomes concrete. If the encoder drove its predictor from the *original* sample while the decoder drives its own from the *reconstructed* one, a single saturated sample would make them diverge, and because DRHE's filters are recursive that divergence would carry through every remaining ramp. The encoder therefore reconstructs exactly what the decoder will see and updates its state from that:

```
int diff_re = wrap_int16(re - pred_re);
if (diff_re == -32768) diff_re = -32767;      // saturate
int recon_re = wrap_int16(diff_re + pred_re); // what the decoder will see
float curr_mag   = hls::sqrt(recon_re*recon_re + recon_im*recon_im);
float curr_phase = hls::atan2(recon_im, recon_re);   // state from recon
```

On real data the reconstruction always equals the original, so the closed loop changes no output bit. Its cost is in the schedule: the reconstruction sits ahead of the polar conversion on the dependency chain, and accounts for 26 of the pipeline's 58 stages.

5.4.2 Wide-integer conversion in the coefficient solver

The LPC solver forms its numerators and determinant as 128-bit integers. The conversion of such a value directly to `float` is not correct above 2^{64} in this toolchain. All three operands are therefore normalised by one common arithmetic right shift into 62 bits before the division, with the shift chosen from the largest of them:

$$k = \max\big(\mathrm{msb}|D|,\ \mathrm{msb}|N_1|,\ \mathrm{msb}|N_2|\big) - 61, \tag{5.1}$$

clamped at zero. Because numerator and denominator are shifted alike, the quotient is unchanged. On ColoRadar the correlations reach only 44 bits, so the shift is zero; on full-scale synthetic input it is what keeps the coefficients correct.

5.5 Decompressors

A decompressor exists for each algorithm, for two reasons. It is what makes bit-exact verification possible — every frame is compressed, decompressed and compared — and it completes the pipeline a vehicle would need, with the encoder at the sensor and the decoder at the central processor. Each decompressor is built twice from the same C++ source: as an FPGA core, and as a PC program compiled through Vitis C-simulation, which uses the same bit-accurate models of the floating-point functions.

5.5.1 Why decoding is the harder half

Entropy decoding is inherently serial: the length of a codeword is known only once it has been matched, and the next codeword starts where it ends. A DRHE decoder that keeps pace with

the compressor — eight values (four channels, real and imaginary) per two clocks — needs eight chained codeword matches and very wide variable shifters per cycle, plus four copies of the floating-point predictor. HLS estimates such a design at about 182,000 LUTs. HLS over-estimates (by about three times for the serial decoder below), but even a third of that would be four times the serial decoder's logic. Keeping pace is also unnecessary: a decoder that takes 2 ms per frame is a hundred times faster than the radar produces frames. The DRHE decoder was therefore designed to be small rather than fast.

5.5.2 DRHE: a two-stage serial decoder

The DRHE decompressor is therefore a dataflow pair of stages connected by a stream:

- **Unpack** decodes one value per clock. It cuts a 29-bit window from two 256-bit input words, matches all sixteen codewords against it in parallel, extracts the payload, and reads the next input word just in time, stopping at TLAST.
- **Rebuild** runs one floating-point predictor that handles one receive channel per step at II = 2 (limited by the ports of the sine/cosine tables), keeps its state in block RAM, and uses exactly the same expressions as the compressor so that its arithmetic matches bit for bit.

The two stages are balanced at about 196,608 clocks per frame, roughly 2 ms at 100 MHz. HLS estimated 42,846 LUTs; after placement and routing the core uses 14,155.

Bit-for-bit agreement of the floating-point arithmetic is not optional for DRHE. The predictor feeds back on itself, so a decoder whose `atan2` or `sin` differs from the encoder's in the last bit will drift. A double-precision Python decoder written as a cross-check is exact for LPC but drifts by 1 to 6 least-significant bits on 8 of the 50 DRHE frames; the decoder built from the HLS source, which the FPGA core shares, is exact on all of them.

5.5.3 LPC

The LPC decompressor reads the coefficient table into registers, then reconstructs each sample as its residual arrives, holding the last $2n_{\mathrm{Tx}}$ reconstructed values of each (channel, bin) in a ring as the compressor does. It needs no frame buffer and no block RAM for data, and its only arithmetic is two fixed-point multiply–accumulates per value.

5.6 The board test system

The cores run in a complete system on an AMD KCU116 evaluation board, whose Kintex UltraScale+ `XCKU5P` provides 216,960 LUTs, 433,920 flip-flops, 480 block RAMs (36 Kb) and 1,824 DSP slices (Figure 5.2).

Hardware. A MicroBlaze processor with 128 kB of local memory controls the system. A DDR4-2400 controller gives access to 1 GB of board memory, which holds the input frames, the compressed frames, the reconstructed frames and the results. An AXI DMA (`dma_c`) streams each frame from DDR4 through the compressor and writes the compressed stream back. In the `_full` build a second DMA (`dma_d`) streams the compressed frame through the decompressor and writes the reconstruction back, so the board compresses, decompresses and checks every frame by itself. A UART, a hardware timer and LEDs complete the system, all on one AXI interconnect at 100 MHz. Each algorithm therefore has two bitstreams: a compress-only build and a `_full` build with the decompressor.

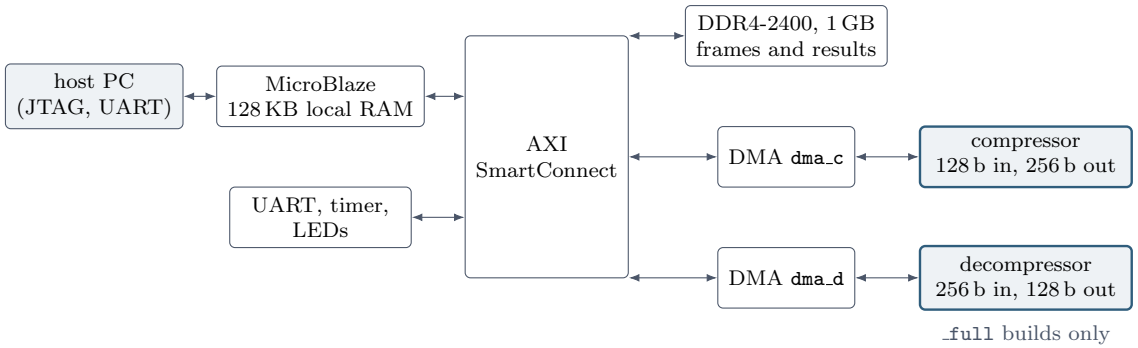


Figure 5.2: The board test system. The host places the frames in DDR4 over JTAG; the MicroBlaze starts the DMAs, times each frame and writes a result record; the host then reads the results and compressed frames back. Everything runs at 100 MHz.

Table 5.1: The four board builds, whole system (processor, DDR4 controller, DMAs and cores), post-route on the `XCKU5P` at 100 MHz, with the re-derived dictionaries; the original-dictionary builds also meet timing. BRAM is in 36 Kb blocks.

Build	LUT	BRAM	DSP	WNS (ns)
DRHE, compress only	61,011 (28 %)	104.5	138	+0.243
LPC, compress only	59,203 (27 %)	162.5	164	+0.174
DRHE, `_full`	81,573 (38 %)	144	184	+0.341
LPC, `_full`	85,707 (40 %)	169.5	172	+0.331

Firmware. For each frame the MicroBlaze configures the core's frame dimensions, starts the DMA transfers, and reads a hardware timer at the start, when the DMA's read channel reports the last input word read, and when its write channel reports the last output word written. From these it records the cycles per frame and the end-to-end latency of Section 4.3, together with the compressed length; the `_full` firmware also records the decompressor's cycles and latency and compares the reconstruction with the original word by word. Each frame's results are a 16-word record in DDR4.

Host. A script on the PC (`run_board.bat`, driving `xsdb`) programs the FPGA, loads the frames into DDR4 over JTAG, runs the firmware, and reads back the result records and compressed frames. For the compress-only build the PC then decompresses every frame with the decompressor built from the HLS source. Finally `compare_results.py` checks every compressed frame byte for byte against the C-simulation output, checks every reconstruction against the original frame, and tabulates compression ratio, cycles and latency.

5.7 Building and timing closure

Each core is exported from Vitis HLS as an IP block and placed in a Vivado block design with the system above; a script builds each of the four bitstreams. HLS estimated both compressors below the 100 MHz target. Its estimate is the tool's own model of operator delays against a budget that reserves about a quarter of the period for clock uncertainty, not a timing analysis of a real circuit, and once placed and routed all four systems meet 100 MHz with positive slack (Table 5.1). Two system details were needed for that: a false path from the DDR4 calibration-complete signal to the processor reset, and the DDR4 interface at 2,400 MT/s.

Chapter 6

Results

This chapter follows the two stages of Section 4.1. It first reports the screening of all eight methods, then breaks down the compression of the two selected algorithms, and then reports their verification and their measured performance on the FPGA board. It closes with the two algorithms side by side.

6.1 Stage 1: screening eight methods

All eight methods were run over the same 50 consecutive ColoRadar frames, with all 16 receive channels and 9,934 reference detections. Table 6.1 and Figure 6.1 give the result.

Table 6.1: Stage 1: all eight methods, 50-frame average, 16 receive channels. "Missed" is the false negative rate and FDR the false discovery rate (Section 4.3); NF is the noise floor. DRHE and LPC predict at lag n_{Tx} and use the original dictionary.

Method	Type	CR	Missed (%)	FDR (%)	NF (dBFS)	SNR (dB)
FP16	reference	1.00	0.00	0.00	−105.64	35.50
FX16	reference	1.00	3.77	1.11	−105.37	35.36
BAQ	lossy	3.76	10.44	9.69	−105.08	35.08
RDVLE	lossy	1.12	24.01	4.62	−102.27	33.17
RDVLE+UDRE	lossy	2.56	28.73	4.44	−101.78	33.00
DRHE fp16	lossless*	1.04	0.05	0.01	−105.64	35.51
DRHE	lossless	**3.75**	3.77	1.11	−105.37	35.36
LPC	lossless	**3.70**	3.77	1.11	−105.37	35.36

*lossless with respect to its FP16 input, which is itself rounded from the reference.

Four observations decide what goes forward.

The lossless coders share FX16's detection metrics exactly. FX16, DRHE and LPC report identical missed-target rates, false discovery rates, noise floors and SNR. This is a consequence of losslessness rather than a coincidence: both coders reconstruct the FX16 codes bit for bit, so the detector sees identical data. What those rows measure is the cost of the *quantisation* step, 3.77 % of targets missed and 1.11 % of detections spurious, none of it the cost of compression.

The lossy methods pay for their ratio in detections. BAQ reaches 3.76, the same ratio as DRHE, but misses 10.44 % of targets instead of 3.77 %, and 9.69 % of what it reports is spurious against 1.11 %: nearly three times the miss rate and nearly nine times the false discovery rate, for no gain in ratio. RDVLE and RDVLE+UDRE are worse on both counts.

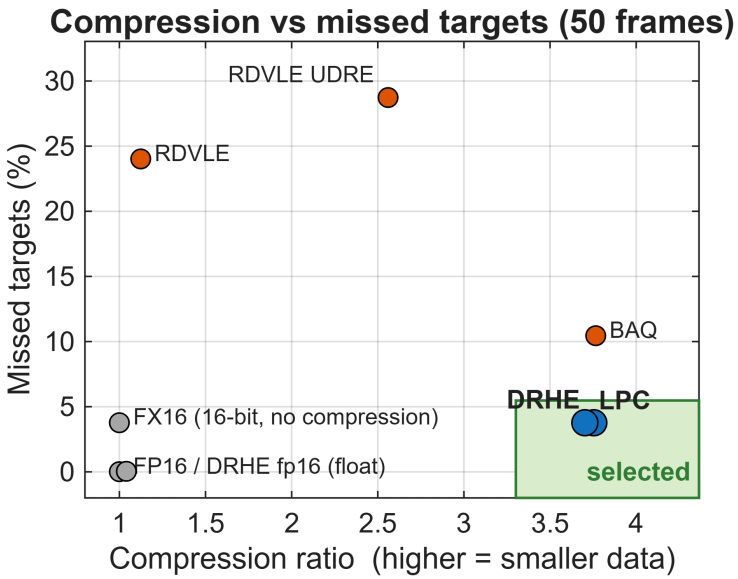


Figure 6.1: Compression ratio against missed targets for the eight methods. A good method sits to the right (small data) and at the bottom (no extra missed targets). Only DRHE and LPC reach the shaded region; the lossy methods pay for their ratio in missed targets, and storing the data as floating point barely compresses.

In a system that brakes a car none of these trades is acceptable, which is the empirical form of the argument of Section 1.3.

Floating-point storage does not compress. DRHE applied to FP16 values almost removes the quantisation loss (0.05 % missed) but compresses by only 1.04. A half-precision mantissa keeps fine noise detail at every magnitude, so the prediction errors never settle near zero. The compressor's *input* must be integer; whether the arithmetic *inside* the predictor is floating point is a separate question, taken up in Section 7.5.

Two methods qualify. DRHE (3.75) and LPC (3.70) are the only methods that compress by more than three without adding a single missed target, and both clear the 2.4–2.7 a Gigabit Ethernet link needs with margin. Both were taken forward to Stage 2 and to hardware.

6.1.1 The cost of the quantisation step

Since FX16 accounts for all the detection loss of the lossless methods, it is natural to ask whether the 16-bit codes could be made more accurate. Table 6.2 compares four ways of producing them on one frame. The standard FX16 scaling, calibrated to a full-scale ADC signal, uses only 1.33 % of the `int16` range: the largest code in the frame is 436. Scaling each frame so that its own peak fills the container cuts the quantisation error by a factor of 67, and the companding curve of Deng and Huang [28] reduces it further.

The compression ratio moves the other way. With the container filled, lossless DRHE compresses by only 1.18; with companding, which spreads the values over the whole code range, it expands the data. Quantisation error and compression ratio are two sides of the same unused bits: the fixed scaling leaves about six of the sixteen bits as copies of the sign bit and rounds away everything below its coarse step, and stretching the data to fill the container turns those six bits into fine detail at the bottom — detail that is mostly receiver noise, which a lossless coder must keep. The quantisation error can be removed, but only by giving up almost all of the compression. The fixed FX16 scaling was therefore kept, as in Kiem's design, and Section 7.3 returns to what this means for the ratios.

Table 6.2: Four ways of producing the 16-bit codes, ColoRadar frame 0, all 16 receive channels. "relRMS" is the RMS error of the range-FFT values relative to their RMS; "RD error" is the RMS error of the range–Doppler map in dB; "int16 use" is the largest code as a fraction of 32,767; the last column is the lossless DRHE ratio on the resulting codes.

Quantiser	relRMS (%)	PSNR (dB)	RD error (dB)	int16 use (%)	DRHE CR
FP16 (reference)	0.0205	101.97	0.0049	—	—
FX16, fixed scaling	1.8636	64.44	0.4336	1.33	**3.80**
FX16, scaled to frame peak	0.0278	101.09	0.0054	89.20	1.18
FX16, companded [28]	0.0048	114.38	0.0011	94.28	0.78

6.2 Stage 2: the prediction axis

Stage 2 uses the four-receiver frames that the hardware processes. The first question is whether the predictors help at all, and the answer depends on the prediction lag (Section 3.3). Table 6.3 compares each predictor against the no-prediction baseline, at lag 1 and at lag n_{Tx}, all with the original dictionary.

Table 6.3: Compression ratio against the no-prediction baseline, 50 frames, four receive channels, original dictionary, 256-bit padding. The predictor rows are HLS C-simulation; all are lossless.

Configuration	CR	vs. no prediction
No prediction (Huffman coding only)	3.479	—
DRHE, lag 1 (adjacent ramp)	3.310	−4.9 %
LPC, lags 1, 2	3.385	−2.7 %
DRHE, lag n_{Tx}	**3.751**	**+7.8 %**
LPC, lags n_{Tx}, $2n_{\mathrm{Tx}}$	**3.697**	**+6.3 %**

Predicting from the adjacent ramp is worse than not predicting at all, for both algorithms: on this radar the adjacent ramp is another transmitter, and the difference between the two is set by the target's bearing, not its velocity. Predicting from the same transmitter turns a 4.9 % loss into a 7.8 % gain for DRHE, and a 2.7 % loss into a 6.3 % gain for LPC. The tail of the error distribution shows the same thing more starkly: the 99th percentile of |residual| is 80.6 with no prediction, 102.4 for DRHE at lag 1, and 15.8 for DRHE and 8.2 for LPC at lag n_{Tx}.

LPC loses less than DRHE on the wrong axis, and the reason is instructive. Yule–Walker chooses the coefficients that minimise the residual energy, and when the previous ramp is another transmitter there is little correlation to exploit, so the fitted coefficients shrink towards zero — and $a_1 = a_2 = 0$ *is* no prediction. Over the 51,200 coefficient sets of the 50 frames, the median fitted a_1 at lags $1, 2$ is -0.02 and half of all values lie between -0.13 and 0.09; at lags $n_{\mathrm{Tx}}, 2n_{\mathrm{Tx}}$ the median is 0.40, with a cluster near 1 from bins holding strong, slowly changing returns. DRHE's coefficients are fixed, so on the wrong axis it goes on extrapolating a phase trend that is not there. A fitted model's worst case is "predict nothing"; a fixed model's worst case is "predict wrongly".

Everything that follows uses lag n_{Tx}.

6.3 Where the compression comes from

Both algorithms share one entropy coder, so their compression ratio can be built up in three steps: what the coder achieves on the FX16 codes alone, what prediction adds, and what a dictionary re-derived for the prediction errors adds. Table 6.4 and Figure 6.2 show the result.

Table 6.4: The compression ratio built up step by step, 50 frames, four receive channels (MATLAB). The "held out" column is the total on frames 5–49 only, which the re-derived dictionary was not trained on.

	Huffman coding only	+ prediction	+ re-derived dictionary	**Total**	Total, held out
DRHE	3.479	+0.272	+0.466	**4.218**	4.207
LPC	3.479	+0.218	+0.707	**4.404**	4.372

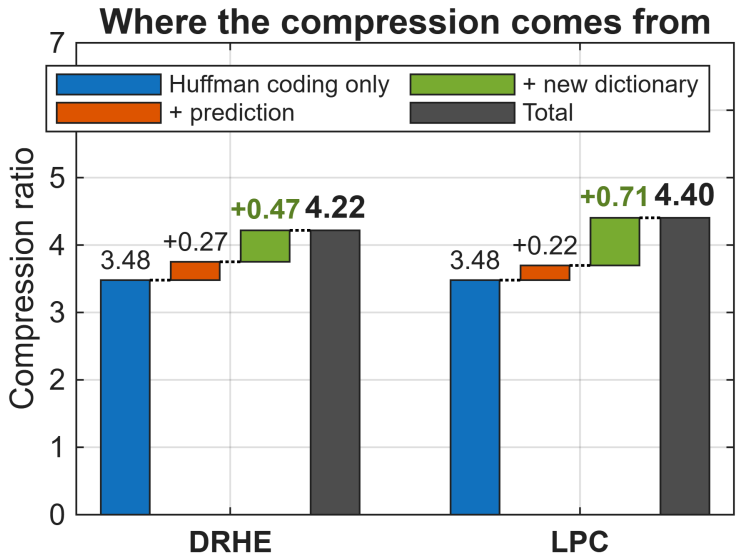


Figure 6.2: The compression ratio built up step by step. Huffman coding alone reaches 3.48; prediction adds 0.22 to 0.27; the re-derived dictionary adds more than prediction does, for both algorithms.

Three things stand out. First, most of the ratio is there before any prediction: the entropy coder alone achieves 3.48. Section 7.3 explains why — largely the unused top bits of the FX16 container, and the sparsity of a range-FFT frame. Second, prediction is worth a modest but real 6 % to 8 %. Third, the dictionary adds more than the predictor does: 0.47 for DRHE and 0.71 for LPC.

Figure 6.3 shows why prediction helps in the currency the coder charges in. The share of values that need zero, one or two bits rises, and the share that need three or more falls; the expensive tail almost disappears.

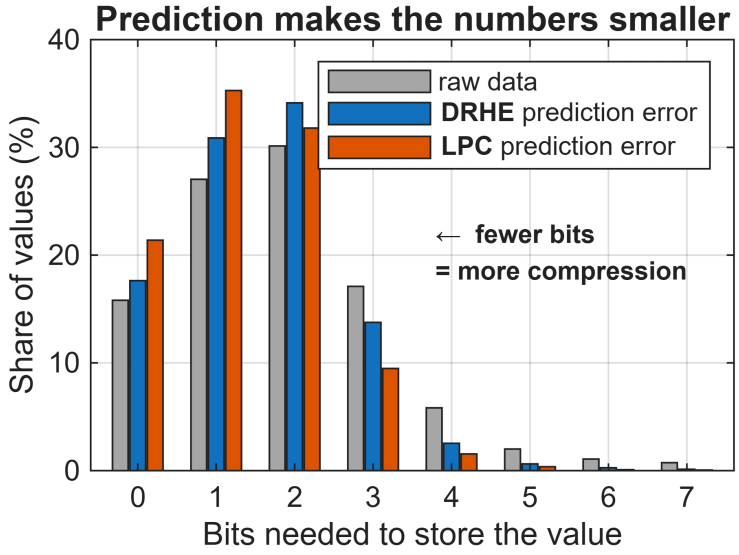


Figure 6.3: How many bits each value needs (S_4), for the raw FX16 codes and for the two algorithms' prediction errors. Prediction moves values towards the cheap end; most prediction errors need two bits or fewer.

6.4 The re-derived dictionary

Table 6.5 sets the share of DRHE's prediction errors in each size category beside the codeword length each dictionary gives it. The original dictionary gives its shortest codes to categories 2 to 4; on this data most errors fall in categories 0 to 2, and categories 3 and 4 hold only 16 %. The re-derived dictionary gives the three commonest categories codes of three, two and one bits, and the average codeword falls from 2.70 to 2.24 bits per value (from 2.80 to 2.12 for LPC).

Table 6.5: Share of DRHE's prediction errors by size S_4, and the codeword length each dictionary assigns (50 frames, four receive channels). Bold marks the three commonest sizes.

Size S_4	0	1	2	3	4	5	Average code
Share of errors (%)	**17.6**	**30.9**	**34.1**	13.8	2.5	0.6	—
Code length, original	4	3	2	2	2	5	2.70 bits
Code length, re-derived	**3**	**2**	**1**	4	5	6	**2.24 bits**

The dictionaries were trained on frames 0–4, so their real test is the 45 frames they did not see. On frames 5–49 DRHE reaches 4.207 and LPC 4.372 (Table 6.4), within 1 % of their all-frame figures: five frames of a scene are enough to fix a dictionary for the rest of the recording. The C-simulation of the FPGA cores with the re-derived tables gives 4.21704 and 4.40345 per frame, matching MATLAB, and reconstructs all 50 frames exactly.

The dictionary also reverses the ranking of the two algorithms. With the original dictionary DRHE leads by 1.5 %; with dictionaries matched to each algorithm's own errors LPC leads by 4.4 %. LPC's errors were always the smaller of the two — over five frames, 4.10 against 4.24 bits per value with the original dictionary, before LPC's 0.17 bits per value of coefficients — but a dictionary that cannot code anything in fewer than four bits hides most of that difference.

6.5 Frame-to-frame variation

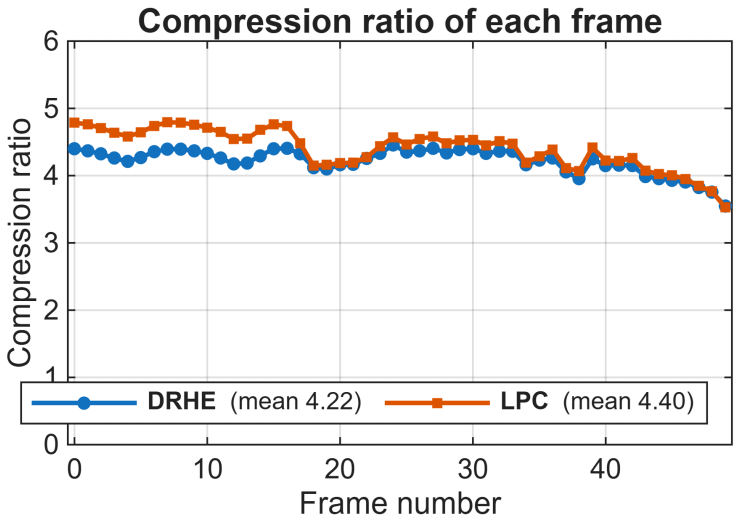


Figure 6.4: Compression ratio of each of the 50 frames, re-derived dictionaries. LPC is ahead on almost every frame; both vary with the scene in the same way.

Figure 6.4 shows the ratio of every frame. It varies by about a third across the recording, from about 3.5 to 4.8, and the two algorithms move together. The variation tracks how much the scene contains: frames with more strong returns have larger prediction errors and compress less, so the per-frame ratio follows the scene rather than any property of the algorithms. For a system that must carry every frame, it is the worst frame, not the mean, that sizes the link; at 3.5 it still leaves a wide margin over the 2.4–2.7 required.

6.6 Verification

Every design passed every level of the ladder of Section 4.4 that it has reached, with MaxDiff = 0:

- **C-simulation**, 50 frames (4,915,200 complex samples, 157,286,400 input bits per design): both compressors, with both the original and the re-derived dictionaries, reconstruct every sample exactly. The bit counts agree with MATLAB on every frame once the zero padding of the last 256-bit output word is counted: MATLAB's LPC model includes it and agrees exactly, and its DRHE model omits it, so the hardware's DRHE counts exceed MATLAB's by 15 to 252 bits, always less than one word.
- **RTL co-simulation**: both compressors reconstruct exactly.
- **Edge cases**: all eight synthetic cases of Table 4.1 pass for both algorithms. Uniformly random input *expands* by 67 % (CR 0.60), as any lossless coder must on incompressible data.
- **Board**: with the original dictionary, both compressors produced compressed output *byte-identical* to the C-simulation on all 50 frames, and every frame decompressed on

the PC with zero error. Mean compression ratio on the board was 3.7508 for DRHE (range 3.517 to 3.815) and 3.6968 for LPC (3.461 to 3.761), exactly the C-simulation figures. The bitstreams with the re-derived dictionaries differ only in the contents of the codeword table and were C-simulated lossless on all 50 frames; **[pending: board run of the re-derived-dictionary bitstreams]**.

- **On-chip decompression**: the `_full` bitstreams, with the decompressor on the FPGA, build and meet timing (Table 5.1); the decompressors were verified lossless on all 50 frames in C-simulation and on the PC with the board's compressed output. **[pending: board run of the on-chip decompression builds]**.

Agreement at the board level is the strongest result here: the bytes produced by the FPGA are the bytes the simulation predicted. The original-dictionary ratios are therefore measured hardware figures, and since the re-derived builds change only the contents of one table, their C-simulated ratios are what the same hardware will produce.

6.7 Speed and latency on the board

Table 6.6 gives the timing measured on the board over 50 frames, and Figure 6.5 draws it.

Table 6.6: Timing measured on the KCU116 at 100 MHz, 50 frames of 393,216 B. Latency is from the last input word read to the last output word written (Section 4.3).

	DRHE	**LPC**
Clock cycles per frame	61,472	325,366
Time per frame	**0.61** ms	3.25 ms
Frames per second	1,627	307
Input throughput	**5.12** Gbit/s	0.97 Gbit/s
End-to-end latency	**13.1 µs**	3,006 µs
as a share of the frame time	2 %	92 %
Decompression time per frame	**[pending: `_full` run]**	**[pending: `_full` run]**

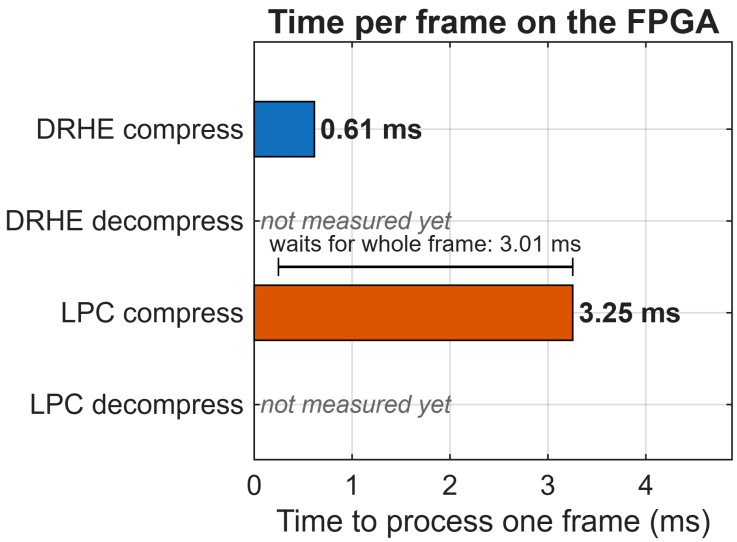


Figure 6.5: Time to process one frame on the FPGA. DRHE outputs as the data arrives; LPC must wait for the whole frame (bracket) before it can output anything.

DRHE streams. Its time per frame, 0.61 ms, is two clocks per input beat (II = 2) plus a short pipeline fill on each ramp. Its latency is 13.1 µs: the last compressed word leaves

almost as soon as the last raw word arrives. The core's own pipeline accounts for about a microsecond of that; the rest is most likely data still in flight in the DMA and interconnect when the last input word is read, though this was not measured separately.

LPC waits. Its time per frame is 3.25 ms, 5.3 times DRHE's, and 92 % of it — 3.0 ms — comes *after* the last input word has arrived, because the coefficients cannot be computed until every ramp of the frame is in. That is 230 times DRHE's latency. It is not an implementation weakness that better scheduling would remove: as long as the coefficients are fitted to the frame being compressed, the first output bit has to wait for the last input sample.

Both are fast in absolute terms. The ColoRadar cascade records at 5 Hz, a frame every 200 ms, so DRHE uses 0.3 % of the frame period and LPC under 2 %. Section 7.2 considers what happens on larger frames.

6.8 Chip resources

Table 6.7 gives the post-route resources of each core, taken from the hierarchical utilisation report of the `_full` builds, and Figure 6.6 draws them as a share of the chip.

Table 6.7: Post-route resources of each core on the `XCKU5P` (216,960 LUT, 433,920 FF, 480 BRAM, 1,824 DSP). BRAM is in 36 Kb blocks.

		LUT	FF	BRAM	DSP
DRHE	compressor	35,444	23,528	40	132
	decompressor	14,155	9,245	32.5	46
	both	**49,599** (23 %)	32,773	72.5	178
LPC	compressor	33,738	20,165	98	158
	decompressor	20,034	8,371	0	8
	both	**53,772** (25 %)	28,536	98	166

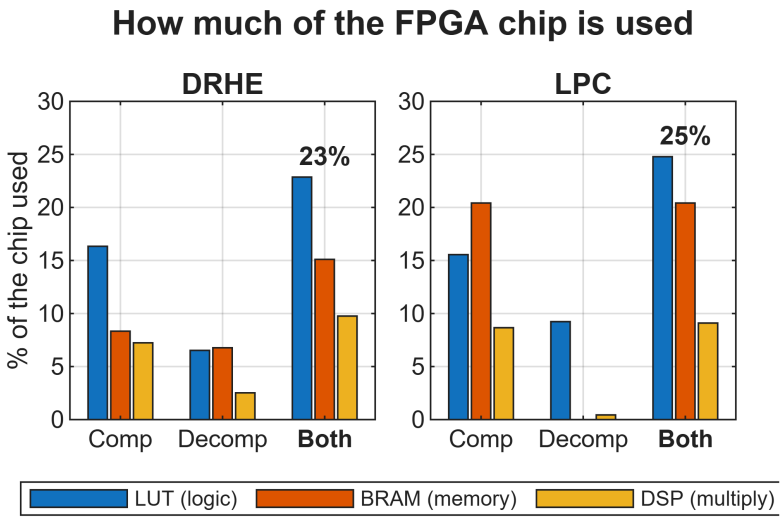


Figure 6.6: Share of the chip used by each core, for the compressor, the decompressor, and both together.

Encoder and decoder together use under a quarter of the chip's logic for both algorithms, and the whole test system — processor, memory controller, DMAs and both cores — uses 38 % (DRHE) and 40 % (LPC) (Table 5.1). The two compressors cost almost the same logic. They differ in memory: LPC's compressor needs 98 block RAMs, most of them its frame

buffer, against DRHE's 40 for predictor state. The decoders invert this. The DRHE decoder carries its own floating-point predictor and its state, while the LPC decoder needs no data memory at all and only eight DSP slices, because the coefficients arrive ready-made in the bitstream.

6.9 The two algorithms side by side

Table 6.8: DRHE and LPC compared on every measured quantity. Bold marks the better of the two on each row.

	DRHE	**LPC**
Lossless (C-sim, 50 frames; board, original dictionary)	yes	yes
Compression ratio, original dictionary (board)	**3.75**	3.70
Compression ratio, re-derived dictionary (C-sim)	4.22	**4.40**
Time per frame	**0.61** ms	3.25 ms
End-to-end latency	**13.1 µs**	3,006 µs
First output available	**immediately**	after the whole frame
Compressor frame buffer	**none**	3.0 Mibit
Encoder + decoder LUTs	**23** %	25 %
Encoder + decoder BRAM	**72.5**	98
Side information	**none**	1.04 %

Table 6.8 collects the results. On compression ratio the two are close, and which one leads depends on the dictionary: DRHE by 1.5 % with the original one, LPC by 4.4 % with dictionaries matched to each. On logic they are level. On time, latency and memory DRHE is clearly ahead: it compresses a frame five times faster, needs no frame buffer, and its output lags its input by microseconds rather than milliseconds. Chapter 7 weighs these against each other.

Chapter 7

Discussion

This chapter weighs the two algorithms against each other for the application that motivates the thesis, considers how the results scale and what the compression ratio really measures, compares the hardware and the results with the baseline, and sets out the limits of what the evidence supports.

7.1 Which algorithm for a sensor front end?

On compression ratio, DRHE and LPC are close, and which one leads depends on the dictionary rather than on the predictors (Section 6.4). On logic they are level. The decisive differences are elsewhere.

Streaming. DRHE's output depends only on samples already received, so it emits compressed data as the raw data arrives, needs no frame buffer, and its output lags its input by 13 µs. LPC's coefficients are sums over every ramp of the frame, so its first output bit has to wait for its last input sample: 3.0 ms of latency and a 3.0 Mibit buffer at this frame size. That is a property of the algorithm, not of this implementation; no amount of scheduling removes it while the coefficients are fitted to the frame being compressed.

Memory. A compressor between the range FFT and the memory bus of a radar sensor has little memory to spare, and LPC's buffer grows with the frame (Section 7.2). DRHE's state also grows, with the number of range bins and transmitters, but it is a fraction of a frame.

Where LPC is attractive. LPC's costs fall on the compressor only. Its decoder is the cheaper of the two — no data memory, eight DSP slices — because the coefficients arrive ready-made. Where latency of a few milliseconds is acceptable and the sensor has memory to spare, LPC's 4 % better ratio is available. A backward-adaptive LPC, fitting its coefficients to *previous* frames instead of the current one, would remove both the buffer and the latency at some cost in ratio, and is a natural next step.

Recommendation. For a streaming sensor-edge compressor, DRHE is the better choice: it is lossless, compresses by 4.2 with a matched dictionary, keeps up with the data with a wide margin, needs no frame buffer, and adds microseconds rather than milliseconds of latency. LPC compresses about 4 % better, but only by waiting for and storing every frame.

7.2 Scaling to larger frames

The hardware path compresses four of the cascade's sixteen receive channels, because four complex 16-bit samples fill the 128-bit stream. The following estimates scale the measured figures linearly; they are not measurements.

- **All 16 receive channels** quadruple the data per frame. DRHE would take about 2.5 ms per frame and LPC about 13 ms, both comfortably within the 200 ms frame period of the ColoRadar capture, but LPC would need a 12 Mibit frame buffer and its latency would grow to about 12 ms.
- **Kiem's use case** — 1024 ramps of 1024 samples, four channels, a 50 ms cycle and a 15 ms compression budget — needs 2.4 Gbit/s at the ADC. After the range FFT each frame is 8.4 MB of complex 16-bit data (512 bins of 16-bit I and Q per ramp and channel), delivered in the 21 ms of acquisition: about 3.2 Gbit/s at the compressor's input. DRHE's measured 5.1 Gbit/s keeps up without any buffer. LPC as built would take about 69 ms per frame, longer than the whole cycle, and would need an 8.4 MB frame buffer against roughly 4.4 MB of block and UltraRAM on this device.

DRHE scales with the data rate; LPC scales with the frame size, and on large frames it runs out of both time and memory. Its time per frame could be cut by about a third by overlapping its passes and amortising the per-bin pipeline fill (Section 5.3), and further only by processing several bins in parallel, but its buffer and its latency would remain.

7.3 What the compression ratio measures

Figure 7.1 follows one value through the compressor, in bits. The 16-bit container starts with 16. Because the FX16 scale is fixed to a theoretical full-scale signal and the ColoRadar capture sits far below it, the largest code over 50 frames is 437, so only $\log_2 437 + 1 = 9.77$ bits ever carry information; the other 6.2 are copies of the sign bit. Removing them is a factor of 1.64, and any entropy coder does it whether or not anything is predicted. The coder then exploits the sparsity of a range-FFT frame — most bins hold noise near zero and a few hold targets — for a further factor of 2.12, reaching 4.60 bits per value. Only then does prediction act, taking DRHE to 4.27 bits (a factor of 1.08), and the re-derived dictionary takes it to 3.79 (1.12).

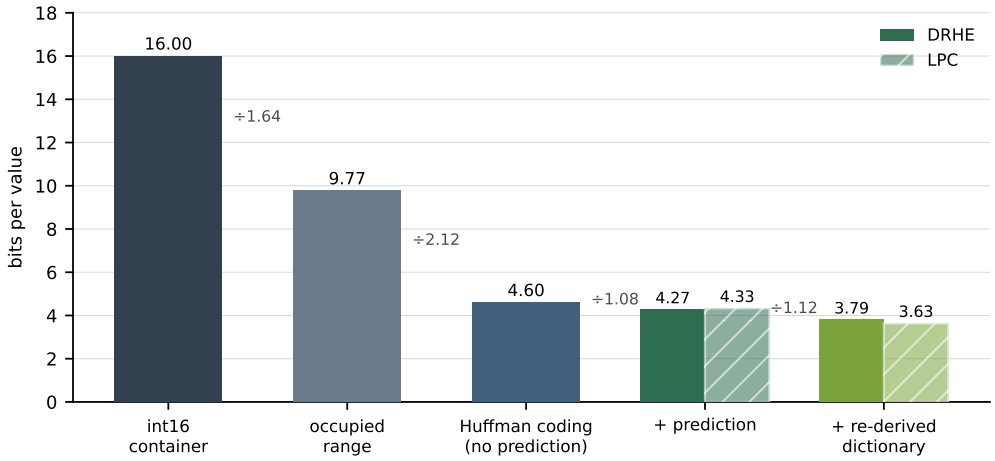


Figure 7.1: Bits per value through the compressor, 50 frames, four receive channels. Most of the reduction comes from the unused top of the 16-bit container and from the sparsity of the data, before any prediction. Factors are for DRHE; LPC is shown hatched.

Two consequences follow.

The ratios are partly a property of the capture. A sensor whose data filled the 16-bit container would have no unused top bits to remove. Its extra bits would instead hold finer detail of the receiver noise, which a lossless coder must keep, and the quantisation study of Section 6.1.1 shows what that does: scaled to fill the container, the same frame compresses by only 1.2. Ratios measured on different sensors are therefore not directly comparable unless their gain staging is. A no-prediction baseline should be reported alongside any radar compression ratio; it costs a few lines of MATLAB and separates the algorithm from the sensor.

The unused bits are a design choice. The same unused bits are why FX16 loses 3.77 % of targets: filling the container removes that loss and, with it, most of the compression (Section 6.1.1). How many bits of the noise floor are worth transmitting is a decision for the radar system designer; a lossless compressor does not change that trade, it only makes whatever choice is made cheaper to carry.

7.4 How far the dictionary and the lag generalise

The re-derived dictionaries were trained on five frames of one recording and tested on 45 later frames of the same recording, where they hold within 1 %. That shows they do not over-fit the training frames; it does not show they transfer to other scenes, sensors or gain settings. Because a dictionary is a sixteen-entry table, a production design could hold several and choose per frame, or update one slowly from the statistics of past frames, without changing anything else in the hardware.

The prediction lag is specific in its value and general in its principle. It is $n_{\mathrm{Tx}} = 12$ because this radar rotates twelve transmitters in a fixed order; a radar that changed its MIMO schedule would need the lag reloaded, and one using Doppler- or code-division multiplexing has no interleaved ramp axis. The principle — a slow-time predictor must predict from the same antenna — applies to any slow-time coder carried over from a single-transmitter design to a time-multiplexed one, and the check is cheap: if the predictor does not beat the no-prediction baseline, it is predicting along the wrong axis.

7.5 Hardware cost against the baseline design

Table 7.1 sets the hardware beside Kiem's. The designs differ in board, data and system, so the comparison is of methods rather than of chips.

Table 7.1: This thesis's hardware against Kiem's [11]. Different boards (Zynq `XCZU3CG` against Kintex `XCKU5P`, both UltraScale+), radars and data. Core resources are the compressor alone; BRAM in 36 Kb blocks.

	Kiem	This work: DRHE	This work: LPC
Lossless CR in hardware	3.17	3.75	3.70
CR, re-derived dictionary (C-sim)	—	4.22	4.40
Hardware checked against simulation	no	byte-identical	byte-identical
Decompressor	none	PC; on chip[a]	PC; on chip[a]
Throughput	11.92 Gibit/s (theor.)	4.77 Gibit/s (meas.)	0.90 Gibit/s (meas.)
End-to-end latency	79.4 µs[b]	13.1 µs	3,006 µs
Compressor LUT / FF	45,892 / 9,243	35,444 / 23,528	33,738 / 20,165
Compressor BRAM / DSP	10 / 44	40 / 132	98 / 158
Arithmetic	fixed point, II = 1	floating point, II = 2	fixed point, II = 1

[a] built and verified in C-simulation; board run pending. [b] includes his FFT core.

Three points need care in reading it.

Throughput units. Kiem's 11.92 Gbit/s is 128 bits per clock at 100 MHz expressed in binary units: $11.92 \times 2^{30}/10^{8} = 128.0$ bits per cycle, the full width of the input stream at $\mathrm{II} = 1$. It is a theoretical rate. The comparable figure for DRHE here is 6.4 Gbit/s at $\mathrm{II} = 2$; measured on the board, including DMA and per-ramp pipeline fill, it achieves 5.12 Gbit/s (4.77 Gibit/s).

Latency. Kiem's 79.4 µs runs from the last DMA read to the last DMA write of a system that includes an FFT core in front of the compressor; the 13.1 µs here uses the same definition without an FFT. The two are the same kind of quantity but not the same pipeline.

Where the logic goes. DRHE here uses fewer LUTs than Kiem's compressor but three times the DSPs and four times the block RAM. To account for the difference, the rebuilt replica and ladder of variants of Section 4.7 were used. Against that replica every difference has one identified cause: the floating-point datapath, above all the arctangent units, accounts for most of the logic, the DSPs and the pipeline depth (58 stages against 12); 32-bit floating-point state words double the block RAM; the per-transmitter state of Section 3.3 doubles it again; and $\mathrm{II} = 2$ comes from one extra write per cycle to reset the state (Section 5.2). Changing only the arithmetic to Kiem's fixed-point types compresses ColoRadar to within 0.07 % of the floating-point design. The floating-point datapath, which Kiem had suggested investigating, is the most expensive design decision in this thesis and buys almost nothing; fixed point is the first improvement to make.

7.6 Comparison with the baseline's results

Kiem reports a ratio of 3.25 in simulation for DRHE; this thesis measures 3.75 with the same dictionary. The two numbers come from different sensors and different data. The rebuilt simulation of Section 4.7 connects them, with three results.

- **His DRHE results are reproduced.** At a noise level of −87.9 dBFS, where the FX16 noise is 4.63 LSB, the rebuilt chain gives 3.25 with run-length coding and 3.24 without, his two figures. His dictionary is optimal for residuals with a standard deviation of 6 to 8 LSB, and his ratio is what a matched dictionary gives on residuals of that size. At the noise level stated for his synthetic scene the same chain gives only 2.28, which confirms that his published ratios come from his recordings, not from that scene.
- **The algorithm is the same.** Running the rebuilt chain and this thesis's DRHE side by side on the same input gives bit-identical output: the DRHE here is Kiem's specification exactly, apart from the prediction lag, which on his single-transmitter radar is 1 because $n_{\mathrm{Tx}} = 1$.
- **Two methods called LPC.** Kiem excluded LPC-Huffman as less promising than DRHE without running it. Run on his scene, that method — fast-time prediction of raw ADC samples — compresses by only 1.32 to 1.54, largely because it codes ADC noise that the FX16 step discards. His judgement of that method stands. The slow-time LPC of this thesis is a different design, and it compresses as well as DRHE.

Taken together, the comparison says that Kiem's DRHE carries over to a twelve-transmitter cascade unchanged except for the prediction lag; that with the lag set correctly and the dictionary matched to the data it compresses public cascade data by 4.2 (a different sensor and scene from his 3.2, so not a like-for-like comparison); and that it can be verified bit for bit on hardware with a decoder, which his system could not.

7.7 Threats to validity

- **The ratios are capture-specific.** Much of the ratio comes from an under-filled container (Section 7.3), a property of this recording's gain staging.
- **One recording.** All results use 50 consecutive frames of one ColoRadar sequence. Scenes with more or faster targets would give larger prediction errors and lower ratios, and the re-derived dictionaries were trained and tested on the same recording.
- **The detection metrics measure the quantisation step.** The three lossless schemes share FX16's detection metrics by construction. The detection chain is also simpler than a production one: its Doppler FFT runs across all 192 ramps in recorded order, it has no angle FFT, detections are matched cell by cell with no positional tolerance, and the threshold is not swept. The absolute rates of the lossy schemes should therefore be read as relative.
- **BAQ is a block-floating-point stand-in.** A better-tuned, statistics-driven BAQ could lose fewer detections at the same ratio; it would still be lossy.
- **Channel count differs between stages.** Stage 1 uses all 16 receive channels; Stage 2 and the hardware use four. The two stages' ratios are close (3.75 and 3.751 for DRHE) but not identical by construction.
- **Some board measurements are outstanding.** The board run with the original dictionary is complete. The re-derived-dictionary bitstreams and the on-chip decompressors are built, meet timing and are verified in simulation, but their board runs, and hence the decompressors' measured time and latency, are **[pending: to be added]**. Power was not measured.
- **Co-simulation covers one frame.** The 50-frame results are from C-simulation and the board, which exercise identical logic; the cycle-accurate RTL simulation was run on one frame.
- **The baseline was rebuilt from its description.** Kiem's source code and recordings are not public. The replica of his hardware and his simulation follow every type, function and parameter his thesis states; the parts it does not state are this thesis's reading of it.

Chapter 8

Conclusions and Future Work

8.1 Summary

This thesis set out to compare data compression methods for SoC-based radar on real, public, multi-transmitter data, to select the best lossless ones, and to implement their encoders and decoders on an FPGA and measure them on a board. The four objectives of Section 1.4 were met, with one measurement still to be made on the board: the decompressors' timing.

Eight methods were compared in MATLAB on 50 frames of ColoRadar data from a twelve-transmitter, sixteen-receiver Texas Instruments cascade. Two lossless predictive coders — Doppler Redundancy Hybrid Encoding and a second-order linear predictive coder — were specified, implemented in C++ for Vitis HLS with matching decompressors, built with Vivado into a test system on an AMD KCU116 board, and verified bit for bit at every level from MATLAB to the board, where the compressors' output matched the simulation byte for byte. A third, independent implementation in Python reproduced their bit counts.

8.2 Findings

1. **Only lossless predictive coding meets the requirement.** DRHE (3.75) and LPC (3.70) are the only methods that compress by more than three without adding a missed target. The lossy quantisers reach up to 3.76 but miss 10 % to 29 % of targets, and floating-point storage compresses by only 1.04. All detection loss in the lossless chain comes from the 16-bit quantisation step, not from compression.
2. **On a TDM-MIMO radar, predict from the same transmitter.** Slow-time prediction from the adjacent ramp, which on this radar is another transmitter, compresses worse than no prediction at all. Predicting $n_{\mathrm{Tx}} = 12$ ramps back, from the same antenna, makes prediction worth 6 % to 8 %. For DRHE the change is an index into the state and nothing else.
3. **Most of the ratio comes before prediction.** Huffman coding alone compresses this data by 3.48, mainly because the fixed 16-bit scaling leaves six bits of every value unused and because a range-FFT frame is sparse. Filling those bits removes the quantisation error but also almost all of the compression. A no-prediction baseline should be reported with any radar compression ratio.
4. **The dictionary matters more than the predictor.** A Huffman dictionary re-derived from five frames for each algorithm's prediction errors raises DRHE from 3.75 to 4.22 and LPC from 3.70 to 4.40, holds on the 45 frames it was not trained on, costs nothing in hardware, and puts LPC ahead of DRHE on ratio.
5. **The hardware is exact and fast.** On the KCU116 at 100 MHz, both compressors produced output byte-identical to the simulation on all 50 frames, and every frame de-

compressed on the PC with zero error. DRHE compresses a frame in 0.61 ms (5.1 Gbit/s) with 13 µs of latency; LPC in 3.25 ms with 3.0 ms of latency, because it must see the whole frame first. Encoder and decoder together use 23 % and 25 % of the chip's logic.

6. **DRHE is the better choice for a streaming sensor.** The two algorithms are close on ratio and level on logic, but DRHE needs no frame buffer, streams its output, and scales with the data rate, while LPC's buffer and latency grow with the frame until, at the frame size of Kiem's use case, it could not keep up.
7. **The baseline carries over, with one change.** Kiem's DRHE is reproduced bit for bit and works on a cascade once the prediction lag is set to the transmitter count; with a matched dictionary it compresses public cascade data by 4.2, and unlike his system it is verified end to end with a decoder.

8.3 Future work

Complete the board measurements. Run the re-derived-dictionary bitstreams and the `_full` bitstreams on the board, to confirm the ratios of 4.22 and 4.40 on hardware and to measure the decompressors' time per frame and latency. Measure power on the board.

Move DRHE to fixed-point arithmetic. The floating-point datapath is the most expensive design decision and buys 0.07 % of ratio. Kiem's fixed-point types would remove most of its logic, DSPs and pipeline depth while remaining exactly lossless, since the closed loop makes the decoder follow whatever arithmetic the encoder uses. With fixed point, reading the state as zero on the first ramps instead of writing it would also reach $\mathrm{II} = 1$ almost for free.

A streaming LPC. A backward-adaptive LPC that fits its coefficients to previous frames would need no frame buffer and add no frame of latency, combining LPC's better prediction with DRHE's streaming. For the forward LPC, overlapping the three passes and amortising the per-bin pipeline fill would cut its time per frame by about a third, and processing several bins in parallel by more.

Dictionaries that adapt. Train dictionaries on more varied scenes, or let the hardware hold several and choose per frame, or update one slowly from past frames. A seventeenth category for the value -32768 would remove the need for saturation.

All sixteen channels and the range FFT on chip. Widen the stream or time-multiplex the cores to compress all sixteen receive channels, and place a range-FFT core in front of the compressor, as in Kiem's system, so that the board processes raw ADC data end to end.

The quantisation trade-off. Study adaptive or companded scaling as a deliberate design choice: how many bits of the noise floor are worth keeping for the detection margin they buy, and what that costs in link bandwidth.

Other multiplexing schemes and methods. Doppler- and code-division MIMO distribute the slow-time structure differently, and whether slow-time prediction helps on them is open. Learned and spectral methods such as AdaRadar [24] define the upper end of achievable ratios and could be compared against the lossless coders on the same data.

Bibliography

[1] C. Waldschmidt, J. Hasch and W. Menzel, "Automotive radar — from first efforts to future systems," *IEEE Journal of Microwaves*, vol. 1, no. 1, pp. 135–148, 2021.

[2] S. Yao *et al.*, "Exploring radar data representations in autonomous driving: a comprehensive review," arXiv preprint, 2023.

[3] M. I. Skolnik, *Radar Handbook*, 3rd ed. New York, NY: McGraw-Hill, 2008.

[4] V. Bandur *et al.*, "Making the case for centralized automotive E/E architectures," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1230–1245, 2021.

[5] Z. Liu, W. Zhang and F. Zhao, "Impact, challenges and prospect of software-defined vehicles," *Automotive Innovation*, vol. 5, no. 2, pp. 180–194, 2022.

[6] I. Barabás *et al.*, "Rethinking vehicle architecture through softwarization and servitization," *IEEE Access*, vol. 12, pp. 110782–110795, 2024.

[7] Texas Instruments, "FPD-Link III serializer/deserializer (SerDes) for sensor applications," Application Note SNLA344, 2021.

[8] Analog Devices, "GMSL SerDes technology for automotive applications," White Paper, 2022.

[9] K. Matheus and T. Königseder, *Automotive Ethernet*. Cambridge, UK: Cambridge University Press, 2021.

[10] IEEE, "IEEE P802.3bp 1000BASE-T1 PHY task force," 2016.

[11] L. Kiem, "Design and implementation of data compression methods for SoC-based radar systems," Master's thesis, Institute of Technical Informatics, Graz University of Technology, Graz, Austria, May 2025.

[12] A. Kramer, K. Harlow, C. Williams and C. Heckman, "ColoRadar: the direct 3D millimeter wave radar dataset," *The International Journal of Robotics Research*, vol. 41, no. 4, pp. 351–360, 2022.

[13] Texas Instruments, *AWR2243 Single-Chip 76- to 81-GHz FMCW Transceiver Datasheet* (Rev. D), February 2024.

[14] K. Sayood, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2017.

[15] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[16] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. Oxford University Press, 2010.

[17] IEEE, "IEEE standard for floating-point arithmetic," IEEE Std 754-2019, 2019.

[18] B. Widrow and I. Kollár, *Quantization Noise*. Cambridge University Press, 2008.

[19] I. Kuon, R. Tessier and J. Rose, "FPGA architecture: survey and challenges," *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.

[20] A. Mani and Z. Yang, "Memory compression and decompression engine for TI mmWave radar," Technical Report, Texas Instruments, 2019.

[21] NXP Semiconductors, "S32R RADAR signal compression," Application Note AN5375, 2021.

[22] R. Kwok and W. T. K. Johnson, "Block adaptive quantization of Magellan SAR data," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 27, no. 4, pp. 375–383, July 1989.

[23] F. Cheng *et al.*, "Autoencoder-based feature compression for radar and vision sensor fusion in autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12345–12355, 2021.

[24] J. Park, S. Y. Chun and M. Seok, "AdaRadar: rate adaptive spectral compression for radar-based perception," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[25] Z. Zivkovic and L. Li, "On-the-fly range-velocity radar data compression exploring Doppler spectrum redundancy," in *Proceedings of the 16th International Radar Symposium (IRS)*, Dresden, Germany, 2015, pp. 321–326.

[26] L. Li, "Embedded data compression in automotive FMCW radar system," MSc thesis, University of Twente, Enschede, The Netherlands, 2014.

[27] G. Meucci and F. Mancuso, "FMCW radar data compression," project report, Data Compression course, PhD programme in Information Engineering, University of Pisa, 2022. [Online]. Available: `https://github.com/mandugo/FMCW-Radar-Data-Compression`

[28] J. Deng and L. Huang, "Synthetic aperture radar image compression based on low-frequency rejection and quality map guidance," *Remote Sensing*, vol. 16, no. 5, art. 891, 2024, doi:10.3390/rs16050891.

Appendix A

The Huffman Dictionaries

Table A.1 gives the complete original dictionary [11, Table A.1], used in Stage 1, in the no-prediction baseline and in the original-dictionary board builds. The published codes are most-significant-bit first; the implementation packs least-significant-bit first, so the stored codewords are bit-reversed. Both forms are listed, and all sixteen entries were verified to agree with the published table.

The reversed code set remains prefix-free, which is what allows the decoder to match candidates from the low end of its bit buffer with no ambiguity.

Table A.1: The original Huffman dictionary. "Cost" is the total bits to code one residual in that category, $\ell + S_4$, from Equation (3.7).

S_4	$\lvert v\rvert$ range	ℓ	code (MSB-first)	stored (LSB-first)	cost
0	$v = 0$	4	`1010`	`0x0005`	4
1	1	3	`100`	`0x0001`	4
2	2–3	2	`01`	`0x0002`	4
3	4–7	2	`00`	`0x0000`	5
4	8–15	2	`11`	`0x0003`	6
5	16–31	5	`10110`	`0x000D`	10
6	32–63	6	`101110`	`0x001D`	12
7	64–127	7	`1011110`	`0x003D`	14
8	128–255	8	`10111110`	`0x007D`	16
9	256–511	9	`101111110`	`0x00FD`	18
10	512–1023	10	`1011111110`	`0x01FD`	20
11	1024–2047	11	`10111111110`	`0x03FD`	22
12	2048–4095	14	`10111111111111`	`0x3FFD`	26
13	4096–8191	14	`10111111111110`	`0x1FFD`	27
14	8192–16383	13	`1011111111110`	`0x0FFD`	27
15	16384–32767	12	`101111111110`	`0x07FD`	27

Note the non-monotonicity at categories 12–15: the longest codewords are assigned to categories 12 and 13 rather than to 15. This is a property of the published dictionary and is reproduced faithfully. It has no effect on this dataset, where no residual exceeds category 10.

The re-derived dictionaries

Table A.2 gives the codeword lengths of the dictionaries re-derived for each algorithm (Section 3.6), built from the S_4 counts of frames 0–4 with one added to every category, and the total cost of each category with each. The codewords themselves are the canonical Huffman codes for these lengths, bit-reversed for the packer; they are the tables `DRHE_TDM_HUFFMAN_TABLE` and `LPC_TDM_HUFFMAN_TABLE` in the HLS source.

Table A.2: Codeword length ℓ and total cost $\ell + S_4$ per category for the original and the two re-derived dictionaries.

	original		DRHE		LPC	
S_4	ℓ	cost	ℓ	cost	ℓ	cost
0	4	4	3	3	3	3
1	3	4	2	3	1	2
2	2	4	1	3	2	4
3	2	5	4	7	4	7
4	2	6	5	9	5	9
5	5	10	6	11	6	11
6	6	12	7	13	7	13
7	7	14	8	15	8	15
8	8	16	9	17	9	17
9	9	18	10	19	10	19
10	10	20	11	21	13	23
11	11	22	14	25	13	24
12	14	26	14	26	13	25
13	14	27	13	26	13	26
14	13	27	13	27	12	26
15	12	27	13	28	12	27

The re-derived dictionaries code their commonest category in three bits (DRHE, category 2) or two (LPC, category 1), where the original cannot code anything in fewer than four. They are longer above category 3, which costs little because those categories are rare after prediction.

Appendix B

Bitstream Layouts

B.1 DRHE

DRHE's bitstream is a single continuous sequence of coded residuals, in the raster order the samples arrive:

$$\underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})}_{\text{ramp 0, bin 0, ch 0}}\ \underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})}_{\text{ramp 0, bin 0, ch 1}}\ \cdots$$

where $\mathrm{enc}(v)$ is the Huffman codeword for $S_4(v)$ followed by $S_4(v)$ APPEND bits. There is no header: the frame dimensions arrive over the AXI4-Lite control interface. The final AXI word is zero-padded.

B.2 LPC

LPC's bitstream has two sections:

$$\underbrace{\left[a_1^{\Re}\ a_2^{\Re}\ a_1^{\Im}\ a_2^{\Im}\right]_{\mathrm{ch,bin}}}_{\text{coefficient table, 4×16 bits each}}\qquad\underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})\ \cdots}_{\text{residuals, raster order}}$$

The coefficient table comes first because the decoder needs all of it before it can reconstruct anything. The residuals follow in the original raster order — not in the order the compressor computed them — which is what allows the decompressor to operate without a frame buffer (Section 5.3).

Appendix C

Reproducing the Results

All sources are in the thesis repository under `ThesisA/`. The build tree must sit on a path without spaces, which Vitis rejects.

C.1 Source files

File	Purpose
MATLAB (`Matlab_Sim/`)	
`loadColoRadarFrame.m`	reads a frame and applies the TDM flattening
`preprocessing.m`, `rangeFFT.m`	DC removal, window, range FFT
`compress_fx16.m`	FX16 quantisation, Equation (2.11)
`compress_drhe.m`, `decompress_drhe.m`	DRHE; lag from `config.predLag`, default n_{Tx}
`compress_lpc_huffman.m`, `decompress_lpc_huffman.m`	LPC at lags L, $2L$
`huffman_tables.m`, `huffman_code_lengths.m`	the original and re-derived dictionaries; retraining
`run_coloradar_benchmark.m`	Stage 1: all eight methods with detection metrics
`make_poster_figs.m`	Stage 2 breakdown, dictionary tables, and the figures of Chapter 6; parameters at the top
`export_frames_for_vitis.m`	writes the `int16` stimulus for the HLS testbenches
HLS (`hls_component/`)	
`drhe_tdm_common.h`, `drhe_tdm_compress.cpp`, `drhe_tdm_decompress.cpp`	DRHE cores and the DRHE dictionary
`lpc_tdm_common.h`, `lpc_tdm_compress.cpp`, `lpc_tdm_decompress.cpp`	LPC cores, Yule–Walker solver and the LPC dictionary
`*_tb.cpp`, `*_tb_edge.cpp`	50-frame and edge-case testbenches
Board (`fpga_kcu116/`)	
`hls/export_ips.bat`	C-simulation (writes the golden streams) and IP packaging
`vivado/build.bat`	block design, synthesis, implementation, bitstream; `decomp 1` adds the decompressor
`sw/src/main.c`, `board_map.h`	MicroBlaze firmware
`host/run_board.bat`, `run_board.tcl`	program the board, load frames, run, read results

`host/pc_decompress.bat`	PC decompression with the HLS decoder source
`host/compare_results.py`	byte comparison with C-simulation, reconstruction check, tables
Independent check (`verify_indep/`, Python)	
`indep.py`	the whole chain from raw frame to bit count
`retrain_complete.py`	dictionaries trained on frames 0–4, tested on 5–49
`golden_check.py`, `independent_decode.py`	residual-level check and decoding of the hardware bitstreams

C.2 Commands

C-simulation, synthesis and co-simulation of a core (DRHE shown):

```
vitis-run --mode hls --csim  --config test_tdm.cfg --work_dir hls_tdm
v++ -c    --mode hls         --config test_tdm.cfg --work_dir hls_tdm
vitis-run --mode hls --cosim --config test_tdm.cfg --work_dir hls_tdm
```

Board run, from `fpga_kcu116/host` (`<tag>` is `drhe`, `lpc`, `drhe_full` or `lpc_full`):

```
run_board.bat <tag> 50
pc_decompress.bat <tag>          (compress-only builds)
python compare_results.py <tag>
```

Figures and tables of Chapter 6, in MATLAB from `Matlab_Sim`: `make_poster_figs`. Simulation results are cached; set `P.recompute = true` after changing the algorithm code.

Appendix D

Verification Record

Table D.1 lists every check on which the results rest, where it was made, and its outcome.

Table D.1: The verification record.

Check	Where	Outcome
Stage 1, eight methods, 50 frames	MATLAB	Table 6.1; re-run reproduces every cell
DRHE and LPC lossless, original dictionary	HLS C-sim, 50 frames	MaxDiff 0 on every frame
DRHE and LPC lossless, re-derived dictionaries	HLS C-sim, 50 frames	MaxDiff 0; CR 4.21704, 4.40345
MATLAB against HLS bit counts	both	equal up to the last 256-bit padding word
Independent bit counts, original dictionary	Python, 50 frames	equal to HLS on every frame
Independent decode, re-derived dictionaries	Python	no residual mismatch
Edge cases, eight synthetic inputs	HLS C-sim	all pass within their allowed MaxDiff
RTL behaviour	co-simulation, 1 frame	MaxDiff 0, both algorithms
Timing closure, four builds	Vivado	all met at 100 MHz (Table 5.1)
Compressed bytes against C-sim, original dictionary	KCU116, 50 frames	byte-identical, both algorithms
Decompression of board output	PC, 50 frames	MaxDiff 0, both algorithms
Board run, re-derived dictionaries	KCU116	**[pending: to be run]**
On-chip decompression (`_full`)	KCU116	**[pending: to be run]**
DRHE matches Kiem's specification	rebuilt simulation	bit-identical output